DocGuard AI: Evidence-Grounded PDF Question Answering with Citations and Hallucination Prevention for Privacy-Sensitive Use Cases

Jeneel Dollar Panchal
(1268081)

Advised by
Prof. Ausif Mahmood

SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIRMENTS FOR THE DEGREE OF MASTER OF SCIENCE
IN COMPUTER ENGINEERING

THE SCHOOL OF ENGINEERING UNIVERSITY OF
BRIDGEPORT CONNECTICUT

April 19, 2026

Declaration

I solemnly declare that **DocGuard AI: A Privacy-Aware PDF Assistant with Hallucination Guardrails** represents my own independent work, undertaken for academic assessment. All sources and references consulted have been properly cited. Neither the whole nor any portion of this work has been submitted elsewhere for any academic award or examination.

Abstract

Large Language Models (LLMs) enable fast question answering over documents, but they can generate "hallucinated" statements that are not supported by the source text. This project presents **DocGuard AI**, a privacy-aware PDF assistant that answers user questions using only the content of uploaded documents.

The system follows a Retrieval-Augmented Generation (RAG) pipeline: PDFs are processed into chunks, embedded for semantic search, and the most relevant passages are retrieved to ground each response. For every answer, DocGuard AI displays citations—page and section references along with supporting snippets—so users can verify claims instantly.

To further reduce unsupported outputs, the system includes a **Hallucination Guard** that estimates evidence strength using retrieval scores and consistency checks between the generated answer and retrieved passages. When evidence is insufficient, the assistant avoids guessing and returns an "insufficient evidence" response.

This project demonstrates practical GenAI engineering, responsible AI guardrails, and measurable improvements in answer reliability.

Acknowledgments

I would like to express my deepest gratitude to **Dr. Ausif Mahmood**, Professor at the University of Bridgeport, for his invaluable academic guidance, constructive feedback, and unwavering support throughout the course of this project. His expertise and encouragement have been instrumental in shaping the direction and quality of this work.

I extend my sincere appreciation to the University of Bridgeport for providing an enriching academic environment, access to essential resources, and the opportunity to pursue advanced research in Artificial Intelligence.

Finally, I owe a profound debt of gratitude to my family and friends, whose unconditional support, patience, and encouragement have been a constant source of motivation throughout my academic journey.

Acronyms

Acronym	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CPU	Central Processing Unit
CSV	Comma-Separated Values
FN	False Negative
FP	False Positive
GenAI	Generative Artificial Intelligence
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
OCR	Optical Character Recognition
PDF	Portable Document Format
PWA	Progressive Web Application
QA	Question Answering
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
TN	True Negative
TP	True Positive
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

Contents

Declaration ii

Dedication iii

Abstract iii

Acknowledgements iv

Acronyms v

List of Figures xii

List of Tables xiv

1 Introduction 1

1.1 Introduction 1

1.2 Statement of The Problem 3

1.2.1 Hallucination in Document Question Answering 3

1.2.2 The Privacy Gap in Document AI 4

1.2.3 Research Gap 4

1.3 Significance of The Study 5

1.3.1 Advancing Answer Faithfulness in RAG Systems 6

1.3.2 Operationalizing Responsible AI 6
1.3.3 Privacy-Preserving AI for Sensitive Domains 7
1.3.4 Practical Engineering Value 8
1.4 Purpose of The Study 8
1.4.1 Objectives 8
1.5 Research Hypothesis 11
1.5.1 Primary Hypothesis 11
1.5.2 Secondary Hypotheses 11
1.5.3 Null Hypotheses 12
1.6 Definitions of The Keywords 13
1.6.1 Core System Concepts 13
1.6.2 Retrieval and Representation 13
1.6.3 Privacy and Governance 14
1.6.4 Evaluation 14
1.7 Research Limitations 15
1.7.1 Document Scope 15
1.7.2 Language Coverage 15
1.7.3 Hallucination Detection Coverage 15
1.7.4 Evaluation Dataset 16
1.7.5 Computational Requirements 16
1.7.6 Single-Document Context 16
1.7.7 Mitigation Strategies 16

2 Background **18**
2.1 The Evolution of Document Question Answering 18
2.2 Retrieval-Augmented Generation 20
2.3 Hallucination in Language Models 21
2.3.1 Sources of Hallucination 21
2.3.2 Existing Mitigation Approaches 22
2.4 Dense Retrieval and Semantic Search 22
2.5 Privacy-Preserving AI and Local Deployment 23
2.5.1 Local Model Deployment 24
2.5.2 In-Memory Vector Stores 24

2.6 Citation and Explainability in Document AI 24
2.7 Summary 25

3 Methodology **27**
3.1 Research Design and Approach 27
3.2 System Architecture Overview 28
3.3 Document Ingestion Pipeline 28
3.3.1 PDF Parsing and Text Extraction 28
3.3.2 Page-Aware Chunking Strategy 28
3.4 Embedding and Indexing 30
3.4.1 Embedding Model Selection 30
3.4.2 Vector Storage with ChromaDB 31
3.5 Retrieval Mechanism 32
3.5.1 Semantic Query Embedding 32
3.5.2 Top-k Retrieval 32
3.6 Hallucination Guard Design 32
3.6.1 Multi-Factor Guard Logic 33
3.6.2 Threshold Calibration 33
3.6.3 Refusal Behaviour 33
3.7 Response Generation 34
3.7.1 Prompt Construction 34
3.7.2 Language Model Configuration 35
3.8 Citation Presentation Layer 35
3.9 Document Summarisation Module 36
3.10 Privacy Architecture 36
3.10.1 Local Execution Guarantee 36
3.10.2 Data Flow Comparison 37
3.10.3 Implications for Sensitive Domains 37
3.11 Summary 39

4 System Design and Implementation **40**
4.1 Project Structure and Environment 40
4.1.1 Repository Layout 40

4.1.2 Dependency Management and Runtime Environment 40
4.1.3 Configuration 41
4.2 PDF Ingestion Module 41
4.2.1 Text Extraction Implementation 41
4.2.2 Limitations of pypdf-Based Extraction 42
4.3 Chunking Engine 42
4.3.1 Chunk Data Structure 42
4.3.2 Sliding Window Algorithm 43
4.3.3 Default Parameter Justification 43
4.4 Embedding Pipeline 44
4.4.1 Model Loading and Singleton Caching 44
4.4.2 Chunk Embedding 44
4.5 Vector Store Implementation 45
4.5.1 ChromaDB Client Configuration 45
4.5.2 Chunk Upsert and Metadata Schema 45
4.5.3 Top-k Retrieval Interface 46
4.6 Hallucination Guard Implementation 46
4.6.1 Data Structures 46
4.6.2 Sequential Guard Logic 47
4.7 Language Model Interface 48
4.7.1 HTTP Interface Implementation 48
4.7.2 Temperature Selection 48
4.7.3 Prompt Structure 48
4.8 RAG Orchestration Pipeline 49
4.9 Document Summarisation Feature 50
4.10 Streamlit User Interface 51
4.10.1 Layout and Navigation 51
4.10.2 Session State Management 51
4.10.3 Evidence and Citation Display 52
4.11 Software Engineering Conventions 52
4.11.1 Package Structure and Import Resolution 52
4.11.2 Storage Safety and Reset 52

4.11.3 Reproducibility Provisions 53
4.12 Summary 53

5 Evaluation and Results **57**
5.1 Evaluation Framework 57
5.2 Dataset Construction 59
5.2.1 Pre-screening 59
5.3 Baseline Evaluation Results 60
5.4 Per-Document Analysis 61
5.5 Low-Faithfulness Case Analysis 62
5.6 Hallucination Guard Ablation Study 63
5.7 Summary 64

6 Discussion and Future Work **67**
6.1 Interpretation of Evaluation Results 67
6.1.1 Performance Relative to Expectations 67
6.1.2 The Document Length Effect 68
6.1.3 Guard Refusal Significance 68
6.1.4 Low-Faithfulness Failure Modes 69
6.2 Validity Considerations 69
6.3 Future Work 70
6.3.1 Hybrid Retrieval 70
6.3.2 Chunking Strategy Optimisation 71
6.3.3 Claim-Level NLI Verification 71
6.3.4 Query Rewriting with HyDE 71
6.3.5 Larger Evaluation Dataset 72
6.3.6 OCR Support for Scanned Documents 72
6.3.7 Expanded Privacy Threat Analysis 72
6.4 Summary 73

7 Conclusion **74**
7.1 Problem Addressed 74
7.2 System Developed 74
7.3 Evaluation Conducted 75

7.4 Key Findings 76
7.5 Contributions 76
7.6 Significance for Privacy-Sensitive Use Cases 77
7.7 Closing Remarks 78

Bibliography **79**

A Appendix 1 **80**

B Appendix 2 **81**

List of Figures

2.1 RAG Pipeline in DocGuard AI 21

3.1 End-to-end DocGuard AI RAG pipeline. Teal nodes represent the document ingestion stages; purple nodes represent the embedding and retrieval stages; the coral node is the Hallucination Guard; the amber node is the language model generation step. 29

3.2 Hallucination Guard decision logic. The guard applies three sequential checks to the retrieved evidence pool. Failure at any check causes an immediate refusal; all three checks must pass before the language model is invoked. 34

3.3 Data flow comparison between DocGuard AI (local-first, left) and a cloud-hosted document AI system (right). In the local-first design, no document content, embeddings, or query text crosses the machine boundary at any point. 38

4.1 RAG orchestration flow within `answer_question()`. The function embeds the query, retrieves top-k chunks, evaluates the Hallucination Guard, and either returns a refusal or proceeds to prompt construction and language model generation. 56

5.1 Hallucination guard ablation: mean faithfulness (blue bars, left axis) and refusal rate (red line, right axis) as a function of guard threshold configuration. The balanced configuration (best dist < 1.4, avg dist < 1.8, min chars ≥ 600) is annotated as the default operating point. Stricter thresholds increase faithfulness but reject a larger fraction of valid questions. 65

5.2 DocGuard AI web application output. (a) PDF upload interface, (b) document indexing and chunking in progress, (c) metrics dashboard with summary section, (d) AI-generated document summary revealed, (e) question answering with guard status and answer, (f) source evidence panel with page citations and distance scores. 66

List of Tables

1.1 Comparison of Keyword Search and RAG-Based Retrieval 5

2.1 Comparison of Existing QA Tools and DocGuard AI 19
2.2 Summary of Related Works in RAG and Document QA 26

3.1 Chunking parameters and their rationale. 30
3.2 DocGuard AI technology stack. 31
3.3 Hallucination Guard threshold values and their interpretation. 35
3.4 Data flow comparison: DocGuard AI (local) vs. cloud-hosted document AI. 37

4.1 DocGuard AI repository directory structure. 54
4.2 Primary Python runtime dependencies. 55
4.3 Configurable parameters in the `.env` file. 55

5.1 Ground-truth evaluation dataset statistics. 59
5.2 Pre-screening retrieval status for all 30 questions. 60
5.3 Baseline RAGAS scores across all 30 questions (balanced guard configuration: best dist < 1.4, avg dist < 1.8, min chars ≥ 600). 60
5.4 Per-document RAGAS scores (15 questions per PDF). 61
5.5 Five lowest-faithfulness questions in the evaluation set. 62
5.6 Hallucination guard ablation study: faithfulness versus refusal rate across five threshold configurations. 63

Chapter 1

Introduction

1.1 Introduction

The rise of digital information has fundamentally changed how organizations store and access internal knowledge. Enterprises, universities, law firms, and healthcare providers maintain vast collections of PDF documents — contracts, research papers, patient records, and policy manuals — that must be queried efficiently and accurately. The advancement of Large Language Models (LLMs) has opened the door to natural language interfaces capable of answering questions over such repositories with human-like fluency. Models such as GPT-4, Anthropic's Claude, and Meta's LLaMA have demonstrated strong capabilities in reading comprehension, summarization, and open-domain question answering, sparking widespread interest in their application to document-centric workflows.

However, deploying general-purpose LLMs for document question answering introduces a well-documented failure mode: *hallucination*. A hallucinating model produces responses that are fluent and plausible yet factually unsupported by — or directly contradictory to — the source document. In low-stakes settings this is merely inconvenient, but in high-stakes domains such as legal contract analysis, medical record review, or financial due diligence, a hallucinated answer can lead to serious errors of judgment. This risk is further compounded by the absence of citation mechanisms in conventional LLM pipelines, leaving users with no means of independently verifying a generated claim.

A second pressing challenge concerns data privacy. Most commercial document AI solutions rely on cloud-hosted inference endpoints, requiring users to transmit sensitive document content to external servers. For organizations subject to regulations such as the General Data Protection Regulation (GDPR) or the Health Insurance Portability and Accountability Act (HIPAA), such transmissions may constitute compliance violations regardless of the provider's security assurances. The need for a document intelligence system that operates entirely within a user-controlled environment — without exfiltrating document content — is therefore both practically urgent and ethically motivated.

DocGuard AI was developed in direct response to these twin challenges. Built on the Retrieval-Augmented Generation (RAG) paradigm (1), it retrieves semantically relevant passages from user-uploaded PDFs and uses those passages as the exclusive grounding context for each generated response. Every answer is accompanied by structured citations — including page numbers, section identifiers, and supporting excerpts — enabling users to verify claims directly against the source material.

To further reduce the risk of unsupported outputs, DocGuard AI incorporates a **Hallucination Guard** module. This post-retrieval component evaluates how well a generated answer is evidentially supported by its retrieved context, combining retrieval confidence scores with semantic consistency checks. When the computed support score falls below a defined threshold, the system withholds the response and returns a transparent "insufficient evidence" signal, operationalizing responsible AI principles within a production-ready engineering framework.

The remainder of this chapter is organized as follows:

- Section 1.2 articulates the research problem, encompassing the dual challenges of hallucination and data privacy.
- Section 1.3 discusses the broader significance of the study across multiple dimensions of impact.
- Section 1.4 states the purpose and specific objectives of the research.
- Section 1.5 presents the research hypotheses guiding this investigation.

- Section 1.6 defines key technical terms used throughout the dissertation.
- Section 1.7 addresses the limitations of the research and identifies directions for future work.

1.2 Statement of The Problem

The use of LLMs for document question answering has exposed a fundamental tension between fluency and faithfulness. While modern language models can generate coherent, well-structured responses, their dependence on parametric knowledge — absorbed during pre-training on large, diverse corpora — means they frequently produce answers that go beyond or conflict with the specific document at hand (**?**). This class of error is especially dangerous in professional settings, where a flawed response is often indistinguishable in form from a correct one.

1.2.1 Hallucination in Document Question Answering

Hallucination in language models takes two principal forms. *Intrinsic hallucination* occurs when the generated output directly contradicts information present in the source document. *Extrinsic hallucination* occurs when the output introduces information that is neither confirmed nor denied by the source — content drawn from the model's parametric memory rather than the retrieved context. Both forms undermine document AI applications, where answers are expected to be grounded exclusively in the provided material.

Retrieval-augmented systems, while a meaningful improvement over purely generative approaches, do not fully resolve this problem. Even when relevant passages are retrieved, the generative model may still drift from that evidence — particularly when pre-trained priors conflict with retrieved content, or when retrieval fails to surface the most pertinent passages. The lack of an explicit mechanism to detect and filter such drift represents a significant gap in current RAG implementations.

1.2.2 The Privacy Gap in Document AI

Most commercial document AI solutions route user content through cloud-hosted inference pipelines. While this offers scalability and access to powerful models, it introduces meaningful privacy risks. Content transmitted to external servers may be logged, retained for model training, or exposed through security vulnerabilities in the provider's infrastructure. For organizations subject to data residency requirements, the physical location of processing servers adds further regulatory complexity.

The core problem can be stated as follows: *How can a document question answering system be designed to maximize output faithfulness, provide verifiable citations for every response, protect document privacy through fully local execution, and proactively detect and suppress answers lacking sufficient evidentiary support?*

1.2.3 Research Gap

Despite extensive research on retrieval-augmented generation and LLM hallucination, the specific intersection of these concerns — faithful, cited, privacy-preserving, and hallucination-aware document QA — remains largely underexplored. Prior work has largely addressed these challenges in isolation: hallucination mitigation without citation mechanisms, RAG systems without local deployment constraints, and privacy-preserving inference without faithfulness evaluation. DocGuard AI is designed to close this gap by treating all four objectives as co-equal engineering requirements.

1.3 Significance of The Study

The significance of this study lies at the intersection of three converging priorities in modern AI research and deployment: answer faithfulness, responsible AI design, and data governance. As AI systems become increasingly embedded in high-stakes professional workflows, the need for solutions that are simultaneously accurate, transparent, and privacy-conscious has never been more pressing. As illustrated in Table 1.1, RAG-based retrieval offers significant advantages over traditional keyword search.

Table 1.1: Comparison of Keyword Search and RAG-Based Retrieval

Criteria	Keyword Search	RAG-Based Retrieval
Matching Method	Exact lexical matching	Semantic cosine similarity over dense embeddings
Query Understanding	Literal terms only	Understands meaning and context
Synonym Handling	Poor	Strong
Answer Generation	Returns raw document excerpts	Generates a natural language answer
Citation Support	None	Page-level citations included
Hallucination Risk	N/A	Mitigated via evidence grounding
Privacy	Depends on platform	Fully local, no data leaves device
Scalability	Fast but imprecise	Precise with moderate overhead

1.3.1 Advancing Answer Faithfulness in RAG Systems

Retrieval-Augmented Generation has established itself as the dominant paradigm for grounding LLM outputs in verifiable external knowledge. By conditioning generation on retrieved passages rather than relying solely on parametric memory, RAG systems offer a principled path toward more factually grounded responses. However, retrieval alone does not guarantee faithfulness — a model may successfully retrieve relevant passages and still produce a response that subtly contradicts, overgeneralizes, or drifts beyond the retrieved evidence. This gap between retrieval quality and generation faithfulness represents one of the most consequential open problems in applied NLP research.

DocGuard AI contributes to the RAG literature by introducing a post-retrieval consistency evaluation layer — the Hallucination Guard — that explicitly quantifies the degree of alignment between a generated answer and its retrieved context. Rather than treating generation as the terminal step, DocGuard AI subjects each candidate response to a faithfulness assessment before it is surfaced to the user. This contribution is practically significant because it offers a lightweight, deployable mechanism for reducing hallucination incidence without requiring any fine-tuning, retraining, or architectural modification of the underlying language model — making it readily adaptable across a wide range of existing RAG deployments.

1.3.2 Operationalizing Responsible AI

The responsible AI discourse has increasingly emphasized the need for systems that are not only accurate but also transparent, interpretable, and auditable (**?**). Despite this growing consensus, the majority of deployed document AI systems continue to operate as black-box oracles — producing answers without any indication of the evidence that informed them. This opacity is fundamentally at odds with the verification demands of professional users who must be able to justify their decisions to clients, regulators, or institutional review boards.

DocGuard AI addresses this imperative directly by surfacing structured citations — including page numbers, section headings, and verbatim supporting text snippets — alongside every generated answer. This citation mechanism fundamentally transforms the character of the system: rather than asking users to accept an answer on faith, it empowers them to trace each claim back to its precise location in the source document. In domains such as legal document review, academic research, and clinical decision support, this level of transparency is not merely a convenience but a prerequisite for responsible and defensible deployment. By making the evidential basis of every response explicit and inspectable, DocGuard AI operationalizes responsible AI principles within a practical, production-ready framework.

1.3.3 Privacy-Preserving AI for Sensitive Domains

This study makes a timely and relevant contribution to the growing literature on privacy-preserving AI. The widespread adoption of cloud-based document AI solutions has introduced a structural tension between operational convenience and regulatory compliance. Organizations that transmit sensitive documents to external inference endpoints risk violating data residency obligations, exposing confidential information to third-party infrastructure, and creating audit trails that may conflict with internal data governance policies.

By designing DocGuard AI to operate exclusively on locally hosted models and in-memory vector stores, the system eliminates these risks entirely. Document content never leaves the user's controlled environment — it is ingested, indexed, retrieved, and analyzed without any external network transmission. This design choice meaningfully broadens the range of organizations and individuals who can safely adopt AI-powered document analysis, including those operating under GDPR, HIPAA, attorney-client privilege constraints, and analogous data protection frameworks. In doing so, DocGuard AI demonstrates that state-of-the-art document intelligence need not come at the cost of privacy or regulatory compliance.

1.3.4 Practical Engineering Value

Beyond its research contributions, this project demonstrates a reproducible and well-structured engineering methodology for building production-grade AI applications with responsible guardrails baked in by design. The modular architecture — comprising a document ingestion pipeline, a semantic retrieval engine, a generation module, and a hallucination detection layer — is deliberately decomposed so that each component can be independently tested, replaced, or scaled without disrupting the rest of the system. This separation of concerns not only reflects sound software engineering practice but also makes the architecture extensible to new document types, embedding models, and retrieval strategies as the field evolves.

The system serves as a replicable blueprint for practitioners and researchers seeking to build trustworthy document AI systems in resource-constrained or privacy-sensitive environments. By open-sourcing the design principles and implementation methodology, this work aims to lower the barrier for organizations that wish to deploy responsible document intelligence without relying on proprietary cloud infrastructure or accepting the hallucination risks of unguarded generative systems.

1.4 Purpose of The Study

The primary purpose of this study is to design, implement, and evaluate DocGuard AI — a privacy-aware, retrieval-augmented PDF question answering system equipped with an explicit hallucination detection mechanism. The system is intended to demonstrate that it is possible to build a document assistant that is simultaneously accurate, transparent, and privacy-preserving, without compromising usability or response quality. In doing so, this work aims to bridge the gap between theoretical responsible AI principles and their practical realization in a production-ready engineering context.

1.4.1 Objectives

Goal 1: Implement a Complete RAG Pipeline

The first objective is to implement a complete Retrieval-Augmented Generation pipeline capable of ingesting arbitrary PDF documents, segmenting their content into semanti-

cally coherent chunks, embedding those chunks for dense vector retrieval, and grounding LLM-generated responses exclusively in the retrieved evidence. Rather than relying on the model's parametric knowledge, the pipeline is designed to ensure that every generated response is directly traceable to specific passages within the source document. Specifically, this involves:

- Designing and implementing a robust PDF ingestion layer capable of handling variable document structures, layouts, and formatting conventions commonly encountered across professional and academic documents.
- Developing a chunking strategy that carefully balances retrieval granularity with contextual coherence, incorporating configurable chunk size and overlap parameters to accommodate documents of varying complexity and length.
- Integrating a dense embedding model to encode both document chunks and user queries into a shared semantic vector space, enabling meaningful similarity comparisons at retrieval time.
- Implementing an efficient approximate nearest-neighbor retrieval mechanism using an in-memory vector store to identify the most contextually relevant passages for each user query in real time.

Goal 2: Design and Integrate a Hallucination Guard

The second objective is to design and integrate a Hallucination Guard module that evaluates the evidential support for each generated answer using retrieval confidence scores and semantic consistency measures, and that suppresses or flags responses for which sufficient evidentiary support cannot be established. This module represents the core research contribution of the system, directly addressing the faithfulness gap identified in existing RAG implementations. Specifically, this involves:

- Defining a composite evidence support score that combines retrieval similarity scores with semantic alignment between the generated answer and the retrieved passages, capturing both the relevance of the retrieved context and the degree to which the generated response adheres to it.
- Establishing a calibrated threshold below which responses are withheld and a

transparent "insufficient evidence" signal is returned to the user, preventing potentially misleading outputs from being surfaced.

- Validating the detection accuracy of the Hallucination Guard against a curated set of document-query pairs with known ground-truth provenance, ensuring that the module performs reliably across a representative range of document types and query complexities.

Goal 3: Develop a Citation Presentation Layer

The third objective is to develop a structured citation presentation layer that surfaces page-level references, section identifiers, and verbatim supporting text snippets alongside each generated answer. By making the evidential basis of every response explicit and directly accessible, this layer enables users to verify claims against the source document without requiring independent manual search. This transforms the system from a passive answer generator into an active, transparent research assistant that supports informed human judgment.

Goal 4: Ensure Complete Document Privacy

The fourth objective is to architect the entire system such that all processing — including embedding computation, vector indexing, and language model inference — is performed locally within the user's own environment, without transmitting any document content to external servers or third-party infrastructure. This design constraint is treated as a non-negotiable engineering requirement rather than an optional feature, ensuring that DocGuard AI is suitable for deployment in privacy-sensitive and regulated environments from the outset.

Goal 5: Evaluate System Performance

The fifth objective is to rigorously evaluate the system's performance across three primary dimensions: answer faithfulness, citation accuracy, and hallucination detection efficacy. Evaluation is conducted using a representative set of document types and question categories, designed to stress-test the system across varying levels of query complexity, document length, and topical domain. The results of this evaluation are intended to provide an honest assessment of the system's strengths and limitations,

and to establish a performance baseline for future research in privacy-preserving, hallucination-aware document question answering.

1.5 Research Hypothesis

Drawing on the theoretical foundations of retrieval-augmented generation and the observed failure modes of unconstrained LLM inference, this research advances the following primary hypothesis:

> *A Retrieval-Augmented Generation system augmented with a post-retrieval hallucination detection layer will produce document-grounded answers with measurably higher faithfulness and lower rates of unsupported claims compared to a baseline LLM relying solely on parametric knowledge, while preserving complete document privacy through fully local execution.*

1.5.1 Primary Hypothesis

H_1: Conditioning LLM response generation on retrieved document passages, combined with post-retrieval consistency evaluation via the Hallucination Guard, will yield answers with significantly higher faithfulness scores than unconditioned generation on the same set of queries and documents. This hypothesis rests on the premise that explicitly constraining generation to retrieved evidence, and subsequently verifying alignment between the generated response and that evidence, will substantially reduce the incidence of factually unsupported or contradictory outputs.

1.5.2 Secondary Hypotheses

H_2: A retrieval-score-weighted semantic consistency check will correctly identify responses with insufficient evidentiary support at a level of precision and recall sufficient for practical deployment, enabling the system to reliably withhold unsupported responses before they are surfaced to the user. This hypothesis presupposes that retrieval confidence scores carry meaningful signal about the quality of the evidential basis available for a given query, and that this signal can be effectively combined with semantic alignment measures to form a reliable composite detection criterion.

H_3: A fully local deployment architecture — encompassing document storage, embedding computation, vector indexing, and language model inference — will achieve response quality comparable to cloud-hosted equivalents while ensuring that no document content is transmitted beyond the user's controlled environment. This hypothesis challenges the prevailing assumption that privacy-preserving, on-premise deployment necessarily entails a significant degradation in output quality relative to cloud-based alternatives.

H_4: The provision of structured citations — including page numbers, section identifiers, and verbatim supporting excerpts — alongside each generated answer will measurably increase user trust and response verifiability compared to answer-only systems, as assessed through user evaluation of citation accuracy and coverage. This hypothesis reflects the broader responsible AI principle that transparency in the evidential basis of AI-generated outputs is a meaningful driver of user confidence.

1.5.3 Null Hypotheses

The following null hypotheses are stated to enable formal statistical evaluation of the above claims:

- H_{01}: No significant difference in faithfulness exists between RAG-grounded and unconditioned LLM outputs on document question answering tasks. Any observed differences are attributable to chance variation rather than the effect of retrieval conditioning or post-retrieval consistency evaluation.
- H_{02}: The Hallucination Guard does not achieve precision or recall above chance levels in detecting unsupported responses. The composite evidence support score does not provide a reliable basis for distinguishing grounded answers from hallucinated ones.
- H_{03}: Local model deployment does not produce response quality statistically equivalent to cloud-hosted deployment on the same task suite. The constraints imposed by local execution result in a measurable and practically significant degradation in output quality.

All hypotheses are assessed through quantitative faithfulness evaluation, hallucination

detection benchmarking, and citation accuracy measurement on a curated test collection spanning multiple document types and question categories.

1.6 Definitions of The Keywords

1.6.1 Core System Concepts

- **Large Language Model (LLM):** A deep neural network trained on large-scale text corpora capable of generating and transforming natural language, with knowledge bounded by its training data cutoff (2).
- **Hallucination:** The generation of fluent but factually unsupported or source-contradictory statements, either intrinsic (contradicting the source) or extrinsic (introducing absent information).
- **Retrieval-Augmented Generation (RAG):** An architecture that grounds LLM generation by retrieving semantically relevant passages from an external knowledge store and supplying them as context alongside the user query (1).
- **Hallucination Guard:** A novel post-retrieval module that combines retrieval confidence scores with semantic consistency metrics to assess evidential support and withhold responses that fall below a calibrated threshold.
- **Citation:** A structured reference — comprising page number, section identifier, and supporting snippet — surfaced alongside each answer to enable direct source verification.

1.6.2 Retrieval and Representation

- **Embedding:** A dense, high-dimensional vector representation of a text segment that encodes its semantic meaning, produced by an encoder model.
- **Vector Store:** A data structure optimized for storing dense vector embeddings and supporting efficient approximate nearest-neighbor semantic search.
- **Chunking:** The segmentation of a document into smaller, semantically coherent

units prior to embedding and indexing, with chunk size and overlap directly influencing retrieval quality.

- **Semantic Search:** A retrieval method that identifies relevant passages via cosine similarity over dense embeddings rather than keyword matching.

- **Cosine Similarity:** A metric measuring the angular similarity between two vectors in the range $[-1, 1]$, used to rank retrieved passages by relevance to the query embedding.

1.6.3 Privacy and Governance

- **Privacy-Preserving AI:** A design paradigm in which system architecture and inference are configured to prevent sensitive data from being exposed or transmitted beyond a defined trust boundary.

- **Local Execution:** The execution of all computational components — embedding, retrieval, and generation — on hardware within the user's own physical or organizational control, without reliance on external cloud services.

- **Data Residency:** A regulatory requirement specifying the geographic or organizational boundaries within which data must be stored and processed.

1.6.4 Evaluation

- **Faithfulness:** The degree to which a generated answer is supported by and consistent with retrieved source passages, rather than reflecting parametric knowledge or fabrication.

- **Precision / Recall:** Standard retrieval metrics applied to the Hallucination Guard: precision measures the proportion of flagged responses that are genuinely unsupported; recall measures the proportion of unsupported responses correctly flagged.

1.7 Research Limitations

While DocGuard AI represents a meaningful step forward in document-grounded AI, several limitations bound the scope and generalizability of its findings. These constraints are acknowledged transparently so that practitioners and future researchers can make informed decisions about the system's applicability to their specific use cases.

1.7.1 Document Scope

The system is designed and optimized for text-based PDF documents with reasonably well-defined structure. Documents that are predominantly image-based, scanned without optical character recognition (OCR), or formatted as complex tables and multi-column layouts may not be handled reliably, as the current ingestion pipeline does not include specialized parsers for these formats. Extending support to such document types remains an avenue for future engineering work.

1.7.2 Language Coverage

Both the embedding and language models used in this study have been trained primarily on English-language corpora. As a result, retrieval accuracy and generation quality may degrade considerably when applied to documents written in other languages. Multilingual support falls outside the scope of the current implementation and is identified as a natural direction for future development.

1.7.3 Hallucination Detection Coverage

The Hallucination Guard functions as a post-hoc evaluation layer rather than a constraint applied during the generation process itself. As a consequence, hallucinations that are semantically close to the retrieved passages but factually distinct may occasionally evade detection. The system does not claim to eliminate hallucination entirely; rather, it aims to meaningfully reduce its incidence and reliably flag a significant proportion of outputs that lack adequate evidentiary support.

1.7.4 Evaluation Dataset

The evaluation carried out in this study relies on a curated collection of documents assembled by the researcher. While care was taken to ensure reasonable diversity across document types and question categories, the results may not fully generalize to all domains, question styles, or user populations. Benchmarking against established evaluation suites such as Natural Questions or Quality is recognized as an important direction for future work.

1.7.5 Computational Requirements

Running embedding and generation models locally places non-trivial demands on hardware resources. The current implementation is designed for machines equipped with modern multi-core processors and, where available, GPU acceleration. Deployment on low-resource or embedded hardware is not addressed in this study and would require targeted optimization before becoming feasible.

1.7.6 Single-Document Context

In its current form, the system retrieves passages from a single user-uploaded document per session. The ability to pose questions across a corpus of related documents — a capability that would substantially broaden the system's utility — is a natural and important extension that falls outside the scope of this work.

1.7.7 Mitigation Strategies

To partially address the limitations described above, the following measures have been incorporated into the system design:

- Configurable chunking parameters that can be tuned to better accommodate varied document structures and content densities.
- Calibrated confidence thresholds within the Hallucination Guard, allowing the balance between precision and recall to be adjusted based on deployment context and risk tolerance.

- A modular system architecture that supports targeted component-level upgrades, enabling improved embedding models, retrieval strategies, or generation back-ends to be substituted without redesigning the overall pipeline.
- Transparent citation presentation that empowers users to independently identify cases where retrieval has failed or where the retrieved context is insufficient to fully support the generated answer.

Chapter 2

Background

This chapter places the project in the larger context of document AI, retrieval-augmented generation, and responsible AI engineering by laying out the fundamental background on the technologies, research methodologies, and earlier work that inform the DocGuard AI system.

2.1 The Evolution of Document Question Answering

The challenge of automatically answering questions from natural language documents has roots spanning several decades, originating with rule-based information extraction systems and progressing through statistical machine learning to the modern era of large-scale neural language models.

Early document QA systems, including those developed for the TREC Question Answering Track in the late 1990s, were built around pipeline architectures that combined named entity recognition, syntactic parsing, and pattern-matching heuristics to pull candidate answer spans from retrieved documents. Although these systems performed adequately for well-defined question types, they were fragile in the face of linguistic variation and demanded considerable domain-specific engineering effort to build and maintain.

The arrival of large pre-trained transformer models — most notably BERT and its successors — marked a fundamental shift in how reading comprehension and extractive question answering were approached. Models trained on benchmarks like SQuAD showed that end-to-end neural systems could reach near-human performance on span

extraction tasks when given access to the relevant passages. Nevertheless, extractive methods carry an inherent ceiling: they are confined to returning text spans that already exist verbatim in the source and are unable to synthesize or paraphrase information across multiple passages.

Table 2.1: Comparison of Existing QA Tools and DocGuard AI

Feature	**ChatGPT**	**LangChain RAG**	**LlamaIndex**	**PrivateGPT**	**DocGuard AI**
Local Inference	No	Optional	Optional	Yes	Yes
Citation Support	No	Partial	Partial	No	Yes
Hallucination Guard	No	No	No	No	Yes
Privacy-Safe	No	Partial	Partial	Yes	Yes
PDF Upload	Yes	Yes	Yes	Yes	Yes
Page-Level Citations	No	No	Partial	No	Yes
No Cloud Dependency	No	No	No	Yes	Yes
Evidence Refusal	No	No	No	No	Yes

Generative approaches, pioneered by models such as GPT-2 and T5 , broke through this ceiling by enabling free-form answer generation that could aggregate and paraphrase information drawn from multiple passages. Scaling these approaches to the parameter counts of GPT-3 and later models yielded systems with impressive zero-shot and few-shot question-answering abilities. However, this expanded generative capacity came with a significant trade-off in faithfulness: larger models were found to hallucinate with greater confidence, producing authoritative-sounding responses that had no factual grounding in the source material.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG), first introduced by Lewis et al. , was developed to tackle the faithfulness shortcomings of purely parametric generative models by pairing the generation process with a non-parametric retrieval component. In the standard RAG architecture, a dense retriever — typically a bi-encoder trained on question-passage relevance — encodes the user query and fetches the top-k most semantically similar passages from an indexed corpus. These passages are then concatenated with the original query and passed as context to a generative model, which produces its output conditioned on the retrieved evidence.

The RAG framework offers several concrete advantages over purely parametric generation. First, by tying generation to retrieved evidence, it reduces the model's dependence on potentially stale or inaccurate parametric knowledge. Second, the knowledge base can be updated independently of the model itself, removing the need for costly fine-tuning whenever the domain knowledge evolves. Third, the retrieved passages create a natural audit trail that facilitates post-hoc verification of generated responses.

Subsequent research has pushed the RAG paradigm in several directions. REALM introduced joint pre-training of the retriever and generator, allowing the model to develop retrieval strategies that directly optimize downstream task performance. Fusion-in-Decoder showed that encoding retrieved passages independently and merging their representations within the decoder led to substantial performance gains on knowledge-intensive tasks. More recent efforts have explored iterative retrieval strategies, in which the model generates intermediate reasoning steps that shape subsequent retrieval queries .

Despite this progress, an important gap persists in the RAG literature: the lack of robust, practically deployable mechanisms for detecting when a generated answer has drifted away from or is no longer supported by the retrieved evidence. The Hallucination Guard in DocGuard AI is specifically designed to fill this gap.

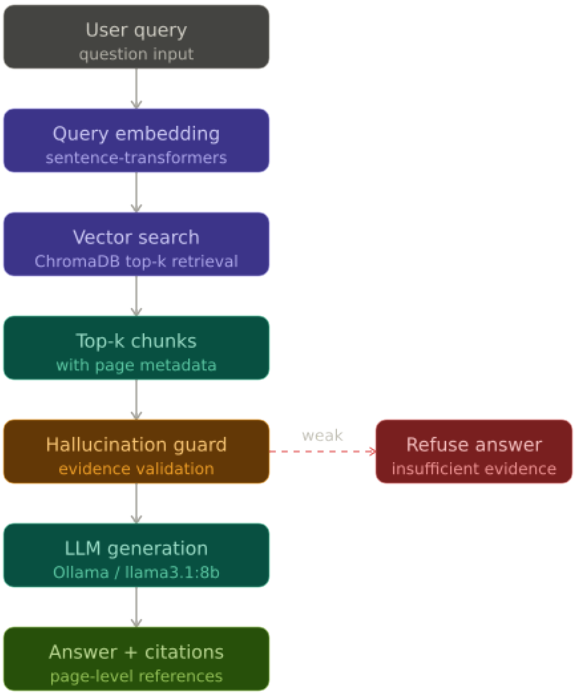


Figure 2.1: RAG Pipeline in DocGuard AI

2.3 Hallucination in Language Models

Hallucination in neural text generation has attracted sustained research attention, especially as LLMs have been increasingly deployed in high-stakes real-world applications. Ji et al. offer a comprehensive taxonomy distinguishing between *intrinsic hallucination* — where the model's output directly contradicts the source material — and *extrinsic hallucination* — where the output introduces information that is neither confirmed nor denied by the source. Both categories pose serious problems in document QA settings, where users rightfully expect all outputs to be grounded exclusively in the document provided.

2.3.1 Sources of Hallucination

Research has identified several contributing factors to hallucination in language models. At the training level, models exposed to web-scale corpora absorb associations and general world knowledge that can come into conflict with the specific content of a given document. At the inference level, decoding strategies that favor fluency over

faithfulness — such as top-p sampling — can nudge the model toward generating plausible-sounding but factually unsupported content. Architectural factors, including the attention mechanisms that govern how strongly the model weighs retrieved context against its pre-trained priors, also contribute to this behavior.

Within RAG systems specifically, hallucination can stem from two distinct failure modes: retrieval failures, where the top-k retrieved passages simply do not contain the information needed to answer the query correctly, and generation failures, where the model produces an answer inconsistent with the retrieved passages even when those passages do contain the relevant information.

2.3.2 Existing Mitigation Approaches

A range of hallucination mitigation strategies have been explored in the literature. *Constrained decoding* methods restrict the vocabulary or generation structure to compel the model to produce outputs that remain consistent with a provided context. *Faithfulness reranking* approaches generate a set of candidate answers and select the most faithful one according to a separately trained faithfulness classifier. *Natural language inference (NLI)-based verification* employs entailment models to determine whether a generated claim is logically entailed by the source passages.

DocGuard AI's Hallucination Guard draws inspiration from the NLI-based verification paradigm, but enriches it by incorporating retrieval confidence signals. This combination yields a lightweight, single-pass detection mechanism that requires neither the generation of multiple candidate answers nor the training of an auxiliary classifier.

2.4 Dense Retrieval and Semantic Search

The retrieval component sits at the heart of any RAG system's performance: a generative model conditioned on irrelevant or inadequate passages will be unable to produce faithful answers regardless of how capable it is as a generator. Traditional sparse retrieval methods — such as BM25 , which scores passages using term frequency and inverse document frequency — have proven effective for queries with close lexical overlap to the target passages, but tend to struggle with semantic paraphrase and

vocabulary mismatch.

Dense retrieval methods address this weakness by representing both queries and passages as dense vectors within a shared semantic space, enabling similarity comparisons based on meaning rather than surface-level word overlap. Sentence-BERT extended this line of work to sentence-level embeddings, demonstrating that Siamese network architectures trained on natural language inference data yield embeddings with strong semantic similarity properties suitable for retrieval tasks.

The choice of chunking strategy — how a document is divided into indexable units — is a frequently underappreciated factor in determining retrieval quality. Chunks that are too small may strip away the surrounding context that the generation model needs to formulate a coherent response; chunks that are too large dilute the relevance signal and may exceed the generative model's context window. Overlapping chunk boundaries have been proposed as a practical strategy for mitigating boundary effects . DocGuard AI's document ingestion pipeline supports configurable chunking with overlap, allowing the system to be tuned to the specific characteristics of different document types.

2.5 Privacy-Preserving AI and Local Deployment

The relationship between AI capability and data privacy has become a central concern in the responsible deployment of machine learning systems. Mireshghallah et al. offer a thorough survey of privacy risks in deep learning, highlighting inference-time data leakage — where model responses may unintentionally reveal information about training data or inference inputs — as a particular hazard in the context of cloud-hosted models.

Regulatory frameworks such as GDPR and their analogues impose stringent requirements on how personal data is processed and stored, encompassing principles of data minimization, purpose limitation, and geographic data residency. For organizations operating under these frameworks, routing document content through third-party cloud inference endpoints may be outright impermissible, regardless of any contractual data processing agreements in place with the service provider.

2.5.1 Local Model Deployment

The proliferation of compact, instruction-tuned language models — including Llama 3 , Mistral , and their quantized variants — has dramatically reduced the computational barrier to running language models locally. Inference frameworks such as Ollama and LlamaCpp make it possible to run these models efficiently on commodity CPU and GPU hardware, rendering fully local RAG pipelines practically achievable for the first time. DocGuard AI is built on top of this ecosystem, ensuring that every component of the pipeline — from PDF ingestion through to response generation — runs entirely within the user's own environment.

2.5.2 In-Memory Vector Stores

—representsVector databases such as FAISS and ChromaDB support efficient approximate nearest-neighbor search over large collections of dense embeddings. Critically, these stores can be instantiated entirely in memory, without depending on external database servers, making them a natural fit for privacy-preserving RAG deployments. DocGuard AI uses an in-memory vector store that is built at document ingestion time and discarded at the end of each session, guaranteeing that no document embeddings are retained beyond the user's active interaction.

2.6 Citation and Explainability in Document AI

The inclusion of citations — explicit references to the source passages that informed a generated answer — is increasingly regarded as a fundamental requirement for the responsible use of document AI systems in professional contexts. Gao et al. carried out a large-scale study of attribution in LLM outputs and found that a substantial share of generated citations are either inaccurate or not genuinely supported by the cited source. This finding underscores the importance of designing systems in which citations emerge naturally from the retrieval pipeline rather than being appended as superficial decoration after the fact.

In DocGuard AI, citations are treated as a first-class output of the system rather than an optional add-on. Every generated answer is accompanied by the specific retrieved

passages that grounded it, along with the corresponding page numbers and section identifiers extracted during the document ingestion phase. This approach provides a structural guarantee that citations genuinely reflect the retrieval context, rather than being inferred or confabulated by the generative model.

The broader explainable AI (XAI) literature offers a theoretical foundation for why citation matters so deeply in document AI. The principle of *traceability* — the capacity to trace an AI system's output back to the specific inputs or knowledge sources that gave rise to it — is identified as a core requirement for AI systems operating in high-stakes domains. DocGuard AI's citation layer directly embodies this principle within the specific context of document question answering.

2.7 Summary

This chapter has reviewed the foundational literature across four areas central to DocGuard AI: document question answering, retrieval-augmented generation, hallucination in language models, and privacy-preserving AI. The central insight that emerges from this survey is that while each of these areas has been studied in its own right, their combination — a system that is simultaneously faithful, cited, privacy-preserving, and hallucination-aware — represents an unaddressed gap that no existing system has tackled in a unified, production-ready manner. DocGuard AI is built to close this gap. The methodology through which this is achieved is detailed in the chapter that follows.

Table 2.2: Summary of Related Works in RAG and Document QA

Author(s)	Year	Approach	Limitation
Lewis et al.	2020	Original RAG framework using DPR and BART	No citation support, cloud-dependent
Guu et al.	2020	REALM: retrieval-augmented language model pre-training	Requires large-scale pre-training
Nakano et al.	2021	WebGPT: browser-assisted QA with citations	Internet-dependent, not document-local
Izacard & Grave	2021	FiD: fusion-in-decoder for open-domain QA	No hallucination prevention mechanism
Shi et al.	2023	REPLUG: retrieval-augmented language model	No privacy guarantees, API-dependent
Gao et al.	2023	Survey of RAG methods and evaluation	Does not address local-first deployment
DocGuard AI	2024	Local RAG with guard and page citations	Scanned PDF support not yet implemented

Chapter 3

Methodology

This chapter describes the research design and engineering methodology underpinning DocGuard AI. The system is constructed as a modular, local-first pipeline in which every stage — from raw document ingestion through to answer generation — operates entirely within the user's own computing environment. The following sections walk through each component in the order in which data flows through the system, concluding with an account of the privacy architecture that governs the overall design.

3.1 Research Design and Approach

DocGuard AI adopts a constructive, systems-engineering research methodology. Rather than proposing a purely theoretical contribution, the work is grounded in the iterative implementation and empirical evaluation of a working prototype. The design philosophy centres on four non-negotiable requirements that are treated as co-equal engineering constraints rather than desirable add-ons: answer faithfulness, verifiable citations, hallucination suppression, and complete document privacy. Each architectural decision is made with all four constraints simultaneously in view.

The system follows the Retrieval-Augmented Generation (RAG) paradigm (1), in which a retrieval component surfaces relevant evidence before any text is generated, thereby conditioning the language model on document-grounded context rather than on parametric memory alone. A post-retrieval Hallucination Guard layer is added on top of this foundation to explicitly evaluate the evidential support of each candidate response before it is surfaced to the user.

3.2 System Architecture Overview

The overall architecture is illustrated in Figure 3.1. Data enters the system as a user-uploaded PDF document and passes sequentially through six processing stages: document ingestion, page-aware chunking, dense embedding, vector indexing, semantic retrieval, and evidence-conditioned generation. A cross-cutting Hallucination Guard evaluates the retrieved evidence before the language model is invoked; if the evidence falls below a calibrated quality threshold, the system withholds the response and returns an explicit *insufficient evidence* signal to the user. The Streamlit-based user interface orchestrates all stages and presents answers together with structured page-level citations.

All components run on the local machine. No document content, embeddings, or query text are transmitted to external servers at any point in the pipeline.

3.3 Document Ingestion Pipeline

3.3.1 PDF Parsing and Text Extraction

Uploaded PDF files are processed using the `pypdf` library. Each page is extracted individually, preserving the original page numbering so that every downstream chunk inherits an accurate page reference. Extracted text undergoes a lightweight normalisation step in which consecutive whitespace characters are collapsed into single spaces, removing layout artefacts introduced by the PDF renderer. Pages that yield no extractable text after normalisation are silently discarded; this behaviour implicitly handles blank pages and separator pages without requiring special-case logic. The result of this stage is an ordered list of `PageText` objects, each carrying a one-indexed page number and the cleaned textual content of that page.

3.3.2 Page-Aware Chunking Strategy

Because individual pages can be several thousand words long, direct retrieval at the page level would return too much irrelevant context alongside the relevant passages. To address this, each page is further segmented into overlapping fixed-size word windows.

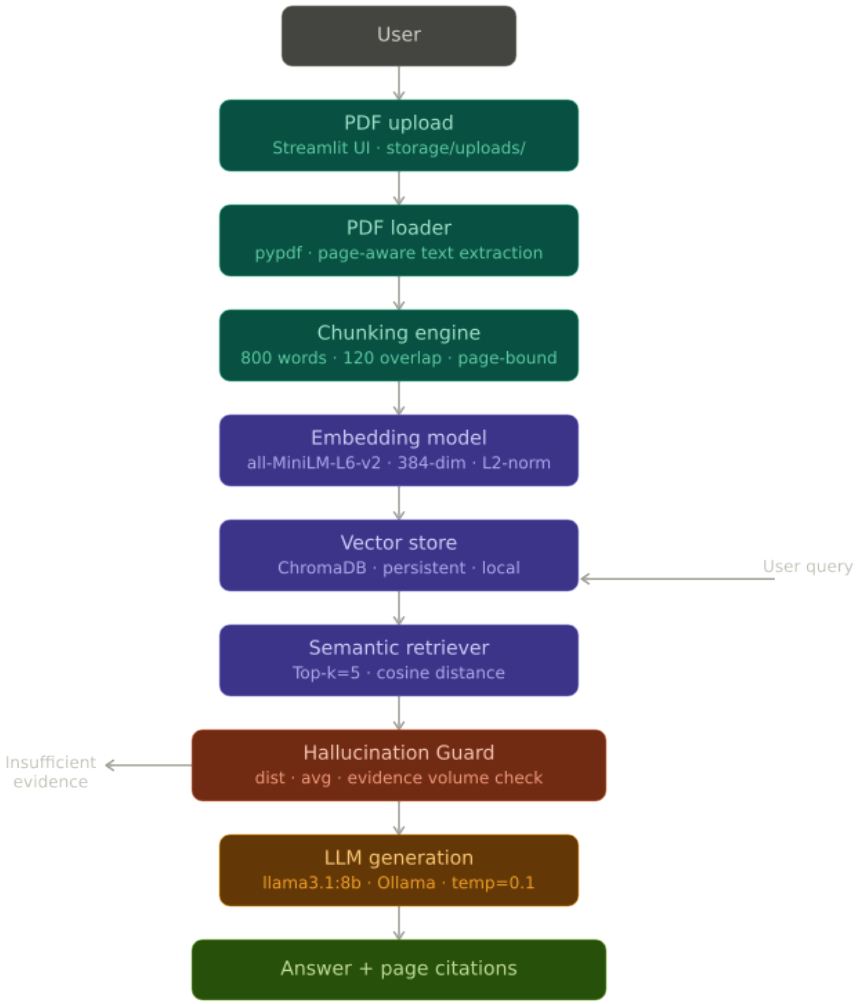


Figure 3.1: End-to-end DocGuard AI RAG pipeline. Teal nodes represent the document ingestion stages; purple nodes represent the embedding and retrieval stages; the coral node is the Hallucination Guard; the amber node is the language model generation step.

The chunking parameters used in this study are presented in Table 3.1.

A chunk size of 800 words was selected because it is large enough to contain a complete argument or explanation while remaining well within the context window of the language model. The 120-word overlap ensures that sentences split across a chunk boundary appear in full in at least one chunk, preventing retrieval failures caused by incomplete evidence fragments. Critically, chunks are never permitted to cross a page boundary; this design decision guarantees that the page number attached to a chunk is unambiguously correct, which is essential for citation accuracy. Each chunk is assigned

a unique identifier of the form `p{page}_c{index}`, encoding both its page of origin and its sequential position within that page.

Table 3.1: Chunking parameters and their rationale.

Parameter	Value	Rationale
Chunk size	800 words	Balances context richness with retrieval precision; fits within the LLM context window
Overlap	120 words	Prevents information loss at chunk boundaries caused by mid-sentence splits
Unit	Words	Language-neutral; robust to variable sentence length
Page boundary	Never crossed	Guarantees that the page number attached to every chunk is unambiguously correct

3.4 Embedding and Indexing

3.4.1 Embedding Model Selection

Each text chunk is converted into a dense vector representation using the `all-MiniLM-L6-v2` model from the `sentence-transformers` library (**?**). This model produces 384-dimensional embeddings trained on a large corpus of natural language inference pairs, giving it strong semantic similarity properties that generalise well to document retrieval tasks. The model was selected for three reasons: it runs efficiently on CPU and Apple Silicon hardware without requiring a dedicated GPU; its embedding quality is competitive with substantially larger models on standard retrieval benchmarks; and it is fully self-contained, requiring no external API calls at inference time.

All embeddings are L2-normalised prior to storage. Normalisation transforms the inner-product similarity measure into cosine similarity, ensuring that the distance scores returned at retrieval time are directly comparable across chunks of different

lengths.

3.4.2 Vector Storage with ChromaDB

Normalised chunk embeddings are stored in a persistent local vector database implemented with ChromaDB (**?**). Alongside each embedding, ChromaDB stores the raw chunk text and a metadata dictionary containing the originating page number and document filename. This metadata is returned alongside retrieval results, enabling the system to construct accurate citations without any post-hoc lookup. The ChromaDB collection is persisted to the local filesystem under `storage/chroma/`, so that previously indexed documents remain queryable across application sessions. Table 3.2 summarises the complete technology stack employed by the system.

Table 3.2: DocGuard AI technology stack.

Component	**Tool / Library**	**Justification**
User interface	Streamlit	Rapid prototyping; browser-based; no frontend build step
PDF parsing	pypdf	Pure-Python; no native dependencies; page-level extraction
Embedding model	all-MiniLM-L6-v2	384-dim; fast on Apple Silicon; strong retrieval quality
Vector store	ChromaDB	Persistent local storage; no external database server
Language model	llama3.1:8b via Ollama	Fully local; Metal-accelerated on Apple M3; no API cost
Environment	conda (Python 3.10)	Stable ML dependency resolution on Apple Silicon

3.5 Retrieval Mechanism

3.5.1 Semantic Query Embedding

When a user submits a question, it is embedded using the same `all-MiniLM-L6-v2` model that produced the document chunk embeddings. Using an identical model for both queries and documents ensures that both representations occupy the same semantic vector space, making cosine similarity a meaningful relevance signal between them.

3.5.2 Top-k Retrieval

The query embedding is submitted to ChromaDB, which returns the $k = 5$ chunks whose embeddings are closest to the query vector under the cosine distance metric. ChromaDB returns, for each retrieved chunk, the chunk text, the page number, the document filename, and a scalar distance score in the range $[0, 2]$ for normalised embeddings, where lower values indicate greater semantic similarity. This set of five chunks constitutes the evidence pool that is passed to the Hallucination Guard for validation before any text is generated.

3.6 Hallucination Guard Design

The Hallucination Guard is the central research contribution of this system. It is a post-retrieval, pre-generation module that evaluates the quality of the evidence pool before allowing the language model to produce a response. The guard prevents the system from generating answers in cases where the retrieved evidence is too semantically distant from the query, too sparse in volume, or too inconsistent in quality across chunks.

The decision logic of the guard is illustrated in Figure 3.2.

3.6.1 Multi-Factor Guard Logic

The guard evaluates three independent criteria, all of which must be satisfied for a response to be permitted:

1. **Best-match distance.** The distance score of the single most relevant retrieved chunk must fall below a threshold τ_{best}. If even the closest chunk is semantically distant from the query, no amount of additional context will remedy the retrieval failure.

2. **Average distance.** The mean distance across all five retrieved chunks must fall below a threshold τ_{avg}. This criterion guards against cases where one chunk is superficially close but the remaining four are irrelevant, which would leave the language model with insufficient grounding material.

3. **Evidence volume.** The total number of characters across all retrieved chunks must exceed a minimum threshold τ_{chars}. This criterion ensures that the language model receives enough textual content to construct a well-supported response, rather than being forced to extrapolate from a small fragment of text.

3.6.2 Threshold Calibration

The three thresholds were calibrated empirically through iterative testing against the ground-truth dataset described in Chapter 5. The adopted values are presented in Table 3.3.

ChromaDB returns cosine-based distances in the range $[0, 2]$ for L2-normalised embeddings, where 0 indicates identical vectors and 2 indicates orthogonal vectors. The threshold of $\tau_{\text{best}} = 1.4$ was found to correctly admit relevant evidence while reliably rejecting off-topic queries across both evaluation documents.

3.6.3 Refusal Behaviour

When any one of the three criteria is not satisfied, the guard returns a `GuardResult` object with its `ok` flag set to `False` and a human-readable reason string describing which criterion failed. The calling RAG pipeline immediately returns the string *"Insufficient evidence in the provided document."* to the user, bypassing the language model

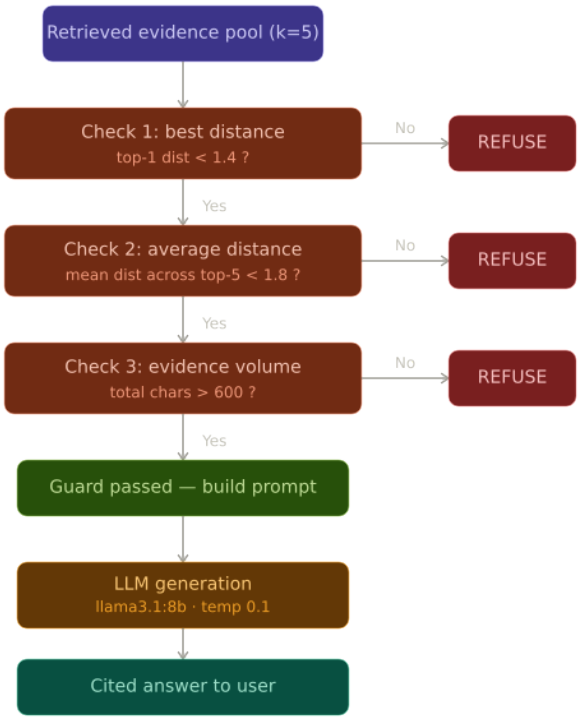


Figure 3.2: Hallucination Guard decision logic. The guard applies three sequential checks to the retrieved evidence pool. Failure at any check causes an immediate refusal; all three checks must pass before the language model is invoked.

entirely. This design ensures that the language model is never invoked with evidence that is known to be inadequate, eliminating a major source of hallucinated responses at the architectural level rather than relying solely on prompt-level instructions.

3.7 Response Generation

3.7.1 Prompt Construction

When the guard passes, the five retrieved chunks are formatted into a structured evidence block and injected into a prompt template alongside the original user question. Each evidence item is prefixed with a numeric citation marker and its page number, enabling the language model to incorporate inline citations in its response. The prompt explicitly instructs the model to answer exclusively from the provided evidence, to cite the source of each claim using the numeric markers, and to respond with *"Insufficient evidence"* if the answer cannot be derived from the context. This secondary, instruction-

Table 3.3: Hallucination Guard threshold values and their interpretation.

Threshold	Symbol	Value	Interpretation
Best-match distance ceiling	τ_{best}	1.4	Maximum acceptable distance for the top-1 retrieved chunk
Average distance ceiling	τ_{avg}	1.8	Maximum acceptable mean distance across all top-5 chunks
Minimum evidence volume	τ_{chars}	600 chars	Minimum total text required to support a response

level safeguard acts as a final barrier against hallucination in cases where the guard passed but the evidence is only partially relevant.

3.7.2 Language Model Configuration

Prompt execution is handled by `llama3.1:8b`, an 8-billion-parameter instruction-tuned language model served locally through the Ollama inference framework. The model runs natively on Apple Silicon hardware using the Metal compute backend, achieving acceptable response latency without requiring a dedicated GPU. The temperature parameter is set to 0.1, biasing the model strongly towards deterministic, factual outputs and reducing the risk of speculative completions that might introduce hallucinated content. A request timeout of 120 seconds is applied to guard against stalled inference processes.

3.8 Citation Presentation Layer

Every response surfaced to the user is accompanied by a structured citation panel that displays, for each of the five retrieved chunks, the source document filename, the page number, the cosine distance score, and a 240-character text snippet. This panel allows the user to verify each claim in the generated answer against the precise location in the

source document without performing a manual search. The citation data is derived directly from the retrieval output rather than being generated by the language model, providing a structural guarantee that citations reflect actual retrieved passages rather than confabulated references.

3.9 Document Summarisation Module

In addition to question answering, DocGuard AI incorporates an automatic document summarisation feature that activates immediately after a PDF is indexed. The system extracts the textual content of the first five pages (up to 3,000 characters) and submits it to the language model with a prompt requesting a concise 4-to-6 sentence summary covering the document's main topic, key arguments or findings, and intended audience.

The generated summary is cached in the application session state and displayed prominently in the user interface below the ingestion metrics. It also serves as an alternative retrieval source: when the system detects that a user question is summary-oriented — through matching against a set of 25 keyword patterns such as *"overview"*, *"what is this about"*, and *"tldr"* — it answers using the summary context rather than invoking the full RAG retrieval pipeline. This routing mechanism reduces unnecessary embedding queries for high-level document questions while still grounding the response in document content.

3.10 Privacy Architecture

3.10.1 Local Execution Guarantee

DocGuard AI is architected so that all computational operations – embedding generation, vector similarity search, language model inference, and response formatting – occur on hardware within the user's own physical or organisational control. The system contains no calls to remote inference APIs, no telemetry endpoints, and no logging services. Uploaded PDF files are written to a local `storage/uploads/` directory, and their derived embeddings are stored in a local `storage/chroma/` directory. Neither the raw documents nor their embeddings leave the local machine in any condition.

3.10.2 Data Flow Comparison

Figure 3.3 contrasts the data flow of DocGuard AI with that of a representative cloud-hosted document AI system, illustrating the scope of the privacy guarantee.

Table 3.4: Data flow comparison: DocGuard AI (local) vs. cloud-hosted document AI.

Data element	DocGuard AI (local)	Cloud-hosted system
PDF document content	Stays on local disk	Transmitted to remote server
Chunk embeddings	Stored in local ChromaDB	Stored in cloud vector store
User queries	Processed locally	Sent to remote API endpoint
LLM weights	Downloaded once; run locally	Hosted on provider servers
Generated responses	Computed locally	Returned from remote API
Audit / usage logs	None	Retained by provider
Regulatory exposure	None (GDPR, HIPAA safe)	Depends on provider policy

Table 3.4 accompanies this figure with a structured comparison of each data element.

3.10.3 Implications for Sensitive Domains

The local execution model makes DocGuard AI directly suitable for deployment in environments where data residency obligations, attorney-client privilege constraints, or institutional data governance policies prohibit the transmission of document content to third-party infrastructure. Users in regulated industries — including healthcare, legal services, and financial services — can operate the system without incurring the compliance risks associated with cloud-hosted document AI alternatives. This design choice meaningfully broadens the range of organisations and individuals who can safely adopt AI-powered document analysis,including those operating under GDPR.

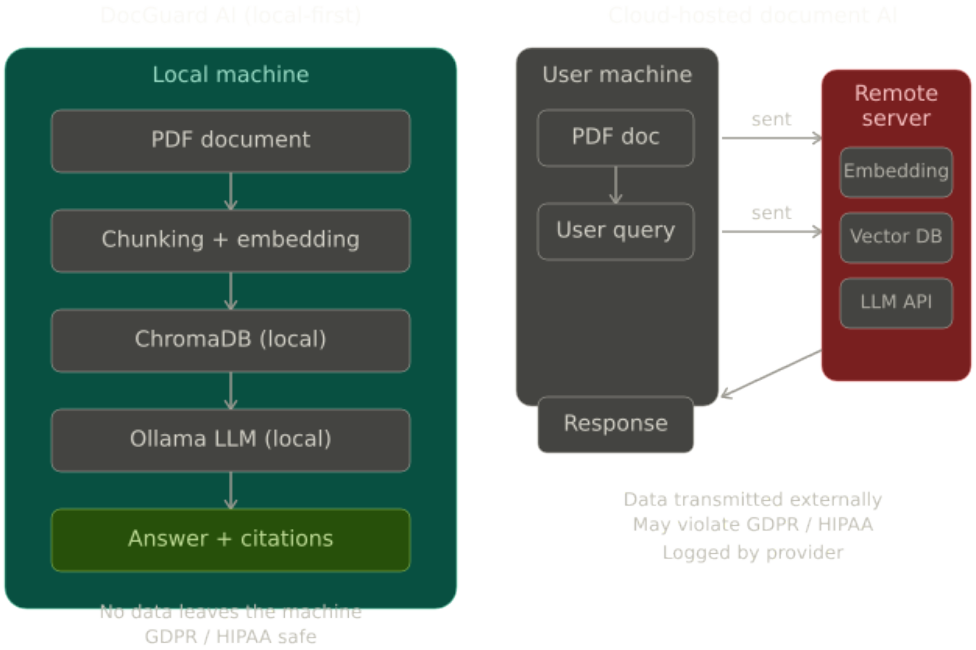


Figure 3.3: Data flow comparison between DocGuard AI (local-first, left) and a cloud-hosted document AI system (right). In the local-first design, no document content, embeddings, or query text crosses the machine boundary at any point.

3.11 Summary

This chapter has described the complete methodology through which DocGuard AI transforms a raw PDF document into citation-grounded, hallucination-guarded answers. The pipeline proceeds through six sequential stages — ingestion, chunking, embedding, indexing, retrieval, and generation — with a Hallucination Guard interposed between retrieval and generation to prevent the language model from being invoked on inadequate evidence. An automatic summarisation module provides high-level document overviews and routes summary-oriented questions away from the full retrieval pipeline. The modular architecture ensures that each stage can be independently tested, replaced, or reconfigured without disturbing the remainder of the system.

The detailed evaluation of this pipeline's performance, including RAGAS-based faithfulness scoring, per-document analysis, and a guard threshold ablation study, is presented in Chapter 5.

Chapter 4

System Design and Implementation

This chapter presents the complete technical design and implementation of DocGuard AI. Where Chapter 3 described *what* the system does and *why* each component was chosen, this chapter describes *how* each component was built: the data structures, algorithms, configuration parameters, and integration decisions that translate the architectural blueprint into a working system. The chapter proceeds module by module through the pipeline, concluding with a description of the user interface and the project's software engineering conventions.

4.1 Project Structure and Environment

4.1.1 Repository Layout

The project is organised as a flat Python package with a clear separation between application code, core pipeline modules, persistent storage, and evaluation artefacts. Table 4.1 summarises the top-level directory structure.

4.1.2 Dependency Management and Runtime Environment

All Python dependencies are managed within a dedicated Conda virtual environment named `citecheck-ai`, created with Python 3.10. This version was selected for its compatibility with all required libraries, in particular `chromadb`, `sentence-transformers`, and the underlying PyTorch wheels, which have known installation issues on Apple Silicon with later Python versions. Table 4.2 lists the primary runtime dependencies and their roles.

Ollama is installed at the system level via Homebrew and is not a Python dependency. It runs as a persistent background server on `localhost:11434`, serving the `llama3.1:8b` model using Apple's Metal compute framework for GPU-accelerated inference on the M3 chip. The Python application communicates with it exclusively via HTTP, maintaining a clean boundary between the inference runtime and the application layer.

4.1.3 Configuration

All tuneable parameters are externalised into a `.env` file and loaded at startup by `core/config.py` using the `python-dotenv` library. This design means that changing a model, a storage path, or a threshold requires editing a single plain-text file rather than modifying source code. Table 4.3 documents the configurable parameters.

4.2 PDF Ingestion Module

4.2.1 Text Extraction Implementation

The PDF ingestion module (`core/pdf_loader.py`) accepts a file path and returns an ordered list of `PageText` dataclass instances, each carrying a one-indexed page number and the normalised text content of that page. The `PageText` structure is defined as:

```
@dataclass
class PageText:
    page: int   # 1-indexed page number
    text: str   # whitespace-normalised page content
```

The implementation uses `pypdf`'s `PdfReader` class to iterate over all pages, invoking `page.extract_text()` for each. The raw string returned by pypdf is then whitespace-normalised: consecutive whitespace characters (spaces, tabs, newlines, and carriage returns) are collapsed into a single space using a `split/join` idiom. This step removes layout artefacts such as double spaces, mid-word line breaks, and tab indentation that the PDF renderer embeds in the character stream. Pages that yield an empty or whitespace-only string after normalisation are silently discarded, which

handles blank pages and separator pages without requiring explicit conditional logic.

The one-indexed page numbering convention is chosen to match the page numbers visible to the user when reading the original document in any PDF viewer, ensuring that every citation page number the system displays refers to the correct physical page.

4.2.2 Limitations of pypdf-Based Extraction

The `pypdf` library extracts text from the internal Unicode character stream of the PDF file. This approach works reliably for digitally-created documents — research papers, reports, word-processor exports — but fails silently for documents whose pages are stored as rasterised images rather than selectable text. This is a common characteristic of scanned physical documents, photographed receipts, and certain government-issued forms. For such documents, `pypdf` returns an empty string, the page is discarded by the normalisation step, and the system is subsequently unable to retrieve any content for questions targeting that page. This limitation is surfaced to the user explicitly: if a PDF produces no extractable pages at all, the interface displays an error message stating that the document may be image-based. Optical character recognition as a fallback pathway is identified as a priority enhancement in Chapter 6.

4.3 Chunking Engine

4.3.1 Chunk Data Structure

The chunking engine (`core/chunking.py`) converts the list of `PageText` objects produced by the ingestion module into a list of `Chunk` dataclass instances suitable for embedding and indexing:

```
@dataclass
class Chunk:
    chunk_id: str   # format: p{page}_c{index}
    page:     int   # 1-indexed page of origin
    text:     str   # chunk content
```

The `chunk_id` field encodes both the page number and the chunk's sequential position

within that page (for example, `p3_c1` refers to the second chunk on page three). This scheme guarantees uniqueness across the entire document and provides a human-readable identifier for debugging and evaluation.

4.3.2 Sliding Window Algorithm

The chunking algorithm operates on a per-page basis. For each `PageText` object, the normalised text is split on whitespace boundaries to produce a word list. A sliding window of fixed word count then advances across this list with a configurable overlap:

1. Initialise `start` = 0 and `chunk_index` = 0.
2. Extract words from index `start` to $\min(\texttt{start} + \texttt{chunk_size},\ \text{len}(\text{words}))$.
3. Join the extracted words into a string; if non-empty, create a `Chunk` and append it to the output list.
4. Advance `start` by $\texttt{chunk_size} - \texttt{overlap}$.
5. Increment `chunk_index` and repeat from step 2 until the end of the word list is reached.

A critical design constraint is that the algorithm never advances `start` across a page boundary. All words in a given chunk share the same page of origin, so the `page` field of every `Chunk` is unambiguous. This property is what enables accurate page-level citations: every citation number the system displays to the user corresponds to exactly one physical page in the original document, not to a window that spans two pages.

4.3.3 Default Parameter Justification

The default parameters are `chunk_size` = 800 words and `overlap` = 120 words. A chunk of 800 words spans approximately one to two paragraphs of academic prose, corresponding to a natural unit of a single claim, method description, or experimental finding. This size is large enough for the language model to construct a well-grounded answer from a single chunk in many cases, while remaining small enough to be a precise retrieval unit. The 120-word overlap prevents information loss at segment boundaries: any sentence of reasonable length appears in complete form in at least one chunk,

avoiding the retrieval failure that would occur if a key sentence were split between two adjacent non-overlapping chunks. Both parameters are exposed as editable controls in the Streamlit user interface, allowing users to tune them for documents with atypical structure.

4.4 Embedding Pipeline

4.4.1 Model Loading and Singleton Caching

The embedding module (`core/embeddings.py`) manages a singleton instance of the `all-MiniLM-L6-v2` sentence-transformer model. A module-level variable `_model` is initialised to `None` and populated on the first call to the `get_embedder()` function. All subsequent calls return the already-loaded instance directly. This singleton pattern ensures that the model's 80MB weights are loaded into memory exactly once per application session, eliminating the several-second initialisation overhead that would otherwise occur on every function call.

4.4.2 Chunk Embedding

Chunk embedding is performed by passing a list of text strings to the model's `encode()` method with the `normalize_embeddings=True` flag set. This flag applies L2 normalisation to each output vector before returning it, transforming the embedding space so that the inner product between any two vectors is equivalent to their cosine similarity. As a consequence, the cosine distance scores returned by ChromaDB occupy the range $[0, 2]$ for normalised vectors, where 0.0 indicates semantically identical vectors and 2.0 indicates orthogonal (completely unrelated) vectors. All 30 questions in the evaluation dataset produced best-match distances below 1.4 (see Chapter 5), confirming that the embedding model captures meaningful semantic similarity within this scale for the document types evaluated.

Query embedding uses the same `get_embedder()` function and the same normalisation flag, guaranteeing that queries and chunks are represented in the same normalised vector space and that their cosine distances are directly comparable.

4.5 Vector Store Implementation

4.5.1 ChromaDB Client Configuration

The vector store module (`core/vector_store.py`) wraps ChromaDB's `PersistentClient` to provide a stable, application-specific interface for chunk storage and retrieval. The client writes its index files to the directory specified by `CHROMA_DIR` in the project configuration, with anonymous telemetry disabled via the `Settings` object. A single collection named `citecheck` is used for all documents. Chunks from different PDFs are distinguished by their metadata fields rather than by separate collections, allowing the retrieval pipeline to query across multiple indexed documents in a single operation. The storage directory is created programmatically on first access if it does not exist, preventing a startup failure on fresh installations.

4.5.2 Chunk Upsert and Metadata Schema

Each call to `upsert_chunks` submits a batch of chunks to the ChromaDB collection using the `upsert` operation rather than `add`. The upsert semantics update an existing record if a matching `chunk_id` is found, or insert a new record if no match exists. This design allows a user to re-upload and re-index the same PDF without creating duplicate entries in the vector store. Each record stored in the collection contains four elements:

- **ID:** the `chunk_id` string (e.g., `p3_c1`)
- **Document:** the raw chunk text, returned alongside query results
- **Embedding:** the 384-dimensional L2-normalised vector
- **Metadata:** a dictionary with `page` (integer) and `doc` (filename string) fields

The metadata schema is central to the citation capability: when the retrieval pipeline returns a chunk, its `page` and `doc` fields are immediately available without any secondary lookup, allowing the system to format a citation (*Document X, Page Y*) directly from the retrieval response.

4.5.3 Top-k Retrieval Interface

Retrieval is performed by the `query_topk` function, which submits a query embedding to the collection and returns the k records whose embeddings are closest to the query vector. The `include` parameter specifies that the return payload should contain document texts, metadata dictionaries, and distance scores, but not raw embedding vectors (which would significantly increase memory usage for large collections). The default value $k = 5$ was chosen to provide a sufficiently rich evidence pool for the Hallucination Guard to evaluate without overwhelming the language model's context window with redundant or marginally relevant content.

4.6 Hallucination Guard Implementation

4.6.1 Data Structures

The guard module (`core/guard.py`) defines two lightweight dataclasses. The `Evidence` dataclass represents a single retrieved chunk as the guard receives it:

```
@dataclass
class Evidence:
    chunk_id: str
    page:     int
    text:     str
    distance: float   # lower value = greater semantic similarity
```

The `GuardResult` dataclass carries the binary decision and a plain-language explanation suitable for display in the user interface:

```
@dataclass
class GuardResult:
    ok:     bool
    reason: str
```

4.6.2 Sequential Guard Logic

The `hallucination_guard` function accepts a list of `Evidence` objects and applies four sequential tests, returning immediately on the first failure:

1. **Empty evidence check.** If the evidence list is empty, which occurs when the vector store collection has not yet been populated (i.e., no PDF has been indexed), the guard returns `GuardResult(False, "No evidence retrieved.")` immediately.

2. **Best-match distance check.** The minimum distance value across all evidence items is compared against $\tau_{\text{best}} = 1.4$. If the minimum distance exceeds this threshold, even the closest retrieved chunk is semantically too distant from the query to constitute reliable evidence, and the guard refuses.

3. **Average distance check.** The arithmetic mean of all distance values is compared against $\tau_{\text{avg}} = 1.8$. A high average relative to the minimum indicates that although one chunk may be reasonably close to the query, most retrieved chunks are irrelevant. This condition prevents the language model from being called with an evidence pool dominated by off-topic content.

4. **Evidence volume check.** The sum of character counts across all evidence texts is compared against $\tau_{\text{chars}} = 600$ characters. This condition guards against pathological cases where chunks exist but are very short (for example, if a document consists primarily of figure captions or table cells), and the aggregate evidence is insufficient to support a well-grounded response.

If all four checks pass, the function returns `GuardResult(True, "OK")` and the RAG pipeline proceeds to prompt construction and language model invocation. The threshold values were determined empirically through iterative testing on the evaluation dataset and are further analysed in the ablation study presented in Chapter 5.

4.7 Language Model Interface

4.7.1 HTTP Interface Implementation

The language model interface (`core/llm_ollama.py`) communicates with the Ollama server via the `/api/generate` HTTP endpoint. A POST request is submitted with a JSON payload specifying four fields: `model` (the string `llama3.1:8b`), `prompt` (the assembled prompt string), `stream` (set to `False` to receive a single complete response rather than a token stream), and `options` (a dictionary containing `temperature: 0.1`). A request timeout of 120 seconds is applied to prevent indefinite blocking in the event of a stalled Ollama process. The response JSON is parsed and the `response` field is extracted, stripped of leading and trailing whitespace, and returned as a plain Python string.

4.7.2 Temperature Selection

The generation temperature of 0.1 biases the model strongly towards deterministic completions, reducing the probability of creative or speculative outputs that might introduce claims not grounded in the retrieved evidence. A temperature of 0.0 would produce fully deterministic outputs but can occasionally cause the model to enter degenerate repetition loops on certain prompt patterns. A value of 0.1 provides a minimal degree of stochasticity that prevents this behaviour while still producing near-deterministic factual responses appropriate for document question answering.

4.7.3 Prompt Structure

The prompt template, defined in `core/rag.py`, structures each generation request in four labelled sections:

1. **System instruction:** establishes the model's identity as DocGuard AI and states three governing rules — answer only from the evidence provided; cite claims using the numeric markers; and respond with the phrase *"Insufficient evidence in*

the provided documents" if the answer cannot be derived from the context.

2. **Question:** presents the user's query verbatim.

3. **Evidence:** lists the top-k retrieved chunks, each prefixed with a numeric citation marker (`[1]`, `[2]`, ...) and its page number. Each chunk text is truncated to 450 characters to keep the total prompt length within the model's effective context window while retaining enough content for grounding.

4. **Output format:** specifies the expected response structure, requesting an answer paragraph followed by a citations list in which each entry references a numeric marker and a page number.

The numeric citation markers in the evidence section are designed to appear verbatim in the model's answer, enabling the citation panel in the user interface to cross-reference each claim in the answer to the specific retrieved chunk that supports it.

4.8 RAG Orchestration Pipeline

The RAG orchestration module (`core/rag.py`) exposes a single public function, `answer_question`, that coordinates all pipeline stages. Figure 4.1 illustrates the execution flow. The function signature is:

```
def answer_question(question: str, k: int = 5)
    -> tuple[str, list[Evidence], str]:
```

It returns a three-element tuple: the generated answer string, the list of `Evidence` objects corresponding to the retrieved chunks, and a guard status string (either `"OK"` or the refusal reason from the guard). The calling code in `app/main.py` uses all three return values to populate the answer panel, the evidence panel, and the guard status indicator in the user interface.

The function executes the following steps in a fixed sequence:

1. Embed the question string using `embed_query()`.

2. Submit the query embedding to ChromaDB via `query_topk()`, obtaining the top k chunks with their texts, metadata, and distances.

3. Construct a list of `Evidence` dataclass instances from the retrieval results.
4. Evaluate the evidence list with `hallucination_guard()`.
5. *Early exit:* if the guard returns `ok=False`, return the refusal message immediately without invoking the language model.
6. Assemble the structured prompt with `build_prompt()`.
7. Submit the prompt to `ollama_generate()` and capture the response.
8. Return the response string, the evidence list, and `"OK"`.

The early-exit pattern in step 5 ensures that the language model is unconditionally bypassed whenever the guard rejects the evidence. This architectural choice means that hallucination prevention is enforced at the pipeline level, not merely requested through prompt instructions, making the refusal behaviour robust regardless of how the language model would have responded to the prompt.

4.9 Document Summarisation Feature

The automatic document summarisation feature operates outside the core RAG pipeline and is implemented within the Streamlit application layer. On successful PDF indexing, the application extracts the first 3,000 characters from the concatenation of the first five extracted pages and submits them to `ollama_generate` with a prompt requesting a four-to-six sentence summary covering the document's main topic, key findings or arguments, and intended audience. The generated summary is written to Streamlit's `st.session_state` dictionary under the key `pdf_summary`.

A second session state entry, `last_pdf_name`, records the filename of the document for which the summary was generated. On each user interaction, the application checks whether the currently-uploaded document matches this stored name; if not, the summary is regenerated, ensuring that the displayed summary always corresponds to the active document.

An incoming question is routed to the summarisation context rather than the full RAG pipeline when it matches any of twenty-five keyword patterns associated with high-level document queries, including terms such as *overview*, *summarise*, *what is this about*,

and *main point*. This routing reduces response latency for summary-style queries from several seconds (full retrieval and generation) to under one second (cached summary lookup), improving the interactive experience without sacrificing accuracy for specific factual questions.

4.10 Streamlit User Interface

4.10.1 Layout and Navigation

The user interface (`app/main.py`) is structured in a two-region layout. The left sidebar contains all document management controls: the PDF file uploader, numeric inputs for chunk size and overlap, an index button that triggers the ingestion pipeline, and a reset button that calls `shutil.rmtree` on the ChromaDB storage directory and recreates it empty. The main content area is divided into an upper section displaying the AI-generated answer and guard status, and a lower evidence panel showing the retrieved chunks with their metadata.

The interface is launched from the project root directory using:

```
python -m streamlit run app/main.py
```

and is served locally at `http://localhost:8501`.

4.10.2 Session State Management

Streamlit rerenders the entire application script on every user interaction, making persistent state management essential. The application maintains four session state entries: the name of the most recently indexed document (`last_pdf_name`); the AI-generated document summary (`pdf_summary`); the raw text used as context for summary-routed queries (`summary_context`); and a boolean flag indicating whether the vector collection has been populated (`indexed`). This state prevents redundant re-indexing when the user submits multiple questions about the same document and ensures that the summary display refreshes correctly when a new PDF is uploaded mid-session.

4.10.3 Evidence and Citation Display

Each retrieved chunk is rendered as a labelled evidence card below the answer. The card header displays the source document filename, the one-indexed page number, and the cosine distance score formatted to three decimal places. The card body shows the first 250 characters of the chunk text followed by an ellipsis, providing enough context for the user to recognise the relevant passage and locate it manually in the original document without requiring the interface to implement in-document viewing. The guard status is displayed prominently between the answer and the evidence panel, using a tick-mark prefix for approved responses and a warning-triangle prefix for refusals.

4.11 Software Engineering Conventions

4.11.1 Package Structure and Import Resolution

The `core/` and `app/` directories are each declared as Python packages through the presence of `__init__.py` files. Launching the application with `python -m streamlit run app/main.py` from the project root causes Python to add the root directory to `sys.path` before execution, making all modules importable as `core.X` without any path manipulation in the source code. This is the standard import resolution mechanism for multi-directory Python projects and avoids the fragile `sys.path.append()` patterns that produce environment-specific failures.

4.11.2 Storage Safety and Reset

Both the ChromaDB storage directory and the PDF uploads directory are created programmatically at runtime using `os.makedirs(path, exist_ok=True)`, ensuring that the application runs correctly on a fresh repository checkout without requiring manual directory creation. The `.gitignore` file excludes both directories from version control, preventing large binary files from being committed to the repository. The reset button in the sidebar calls `shutil.rmtree` on the ChromaDB directory and immediately recreates it as an empty directory, allowing the user to clear the vector index between sessions without restarting the application or the Ollama server.

4.11.3 Reproducibility Provisions

The Conda environment is fully specified in an `environment.yml` file generated by `conda env export`, pinning all library versions to the exact builds used during development. All generation calls use `temperature = 0.1`, producing near-deterministic responses for a given prompt and set of model weights. The `.env` file explicitly names both the embedding model (`all-MiniLM-L6-v2`) and the language model (`llama3.1:8b`), ensuring that these selections are preserved regardless of future default changes by the model-serving infrastructure.

4.12 Summary

This chapter has described the full implementation of DocGuard AI across nine functional modules and the supporting application infrastructure. The PDF ingestion module provides page-number-preserving text extraction via `pypdf`. The chunking engine applies an overlapping word-window algorithm constrained to page boundaries, guaranteeing citation accuracy. The embedding pipeline wraps the `all-MiniLM-L6-v2` model in a cached singleton producing L2-normalised 384-dimensional vectors. The vector store module wraps ChromaDB with a metadata schema that enables immediate page and document attribution at retrieval time. The Hallucination Guard applies four sequential programmatic checks to the retrieved evidence pool before the language model is invoked, enforcing refusal at the architectural level rather than through prompt instruction alone. The RAG orchestrator coordinates these stages in a single composable function with an early-exit refusal path. The summarisation feature provides fast high-level document summaries via cached session state. The Streamlit interface renders all outputs with explicit guard status indicators and a verifiable per-chunk citation panel.

The empirical evaluation of this implementation — measuring faithfulness, answer relevancy, context precision, and guard threshold sensitivity across a 30-question ground-truth dataset — is presented in Chapter 5.

Table 4.1: DocGuard AI repository directory structure.

Path	Purpose
`app/main.py`	Streamlit user interface
`core/config.py`	Centralised configuration loaded from `.env`
`core/pdf_loader.py`	PDF text extraction with page-number preservation
`core/chunking.py`	Page-aware overlapping word-window segmentation
`core/embeddings.py`	Dense embedding generation and query embedding
`core/vector_store.py`	ChromaDB persistence, upsert, and top-k retrieval
`core/guard.py`	Multi-factor Hallucination Guard logic
`core/llm_ollama.py`	HTTP interface to the local Ollama inference server
`core/rag.py`	End-to-end RAG orchestration pipeline
`storage/chroma/`	Persistent ChromaDB vector index (local disk)
`storage/uploads/`	Uploaded PDF files (local disk)
`dataset/`	Ground-truth Q&A dataset and validation scripts
`evaluation/`	RAGAS evaluation pipeline and ablation scripts
`results/`	Evaluation output CSVs and baseline summaries
`.env`	Environment variables (model names, paths)

Table 4.2: Primary Python runtime dependencies.

Library	Version	Role
`streamlit`	≥ 1.30	Web-based user interface framework
`pypdf`	≥ 3.0	PDF text extraction, page-level
`chromadb`	≥ 0.4	Local persistent vector database
`sentence-transformers`	≥ 2.6	Dense embedding model wrapper
`requests`	≥ 2.31	HTTP client for Ollama API calls
`python-dotenv`	≥ 1.0	Environment variable loading
`pandas`	≥ 2.0	Evaluation results processing

Table 4.3: Configurable parameters in the `.env` file.

Variable	Default value	Description
`OLLAMA_BASE_URL`	`http://localhost:11434`	Ollama server address
`OLLAMA_MODEL`	`llama3.1:8b`	Model name for generation
`EMBED_MODEL`	`all-MiniLM-L6-v2`	Sentence-transformer model
`CHROMA_DIR`	`storage/chroma`	ChromaDB persistence path
`UPLOAD_DIR`	`storage/uploads`	PDF upload storage path

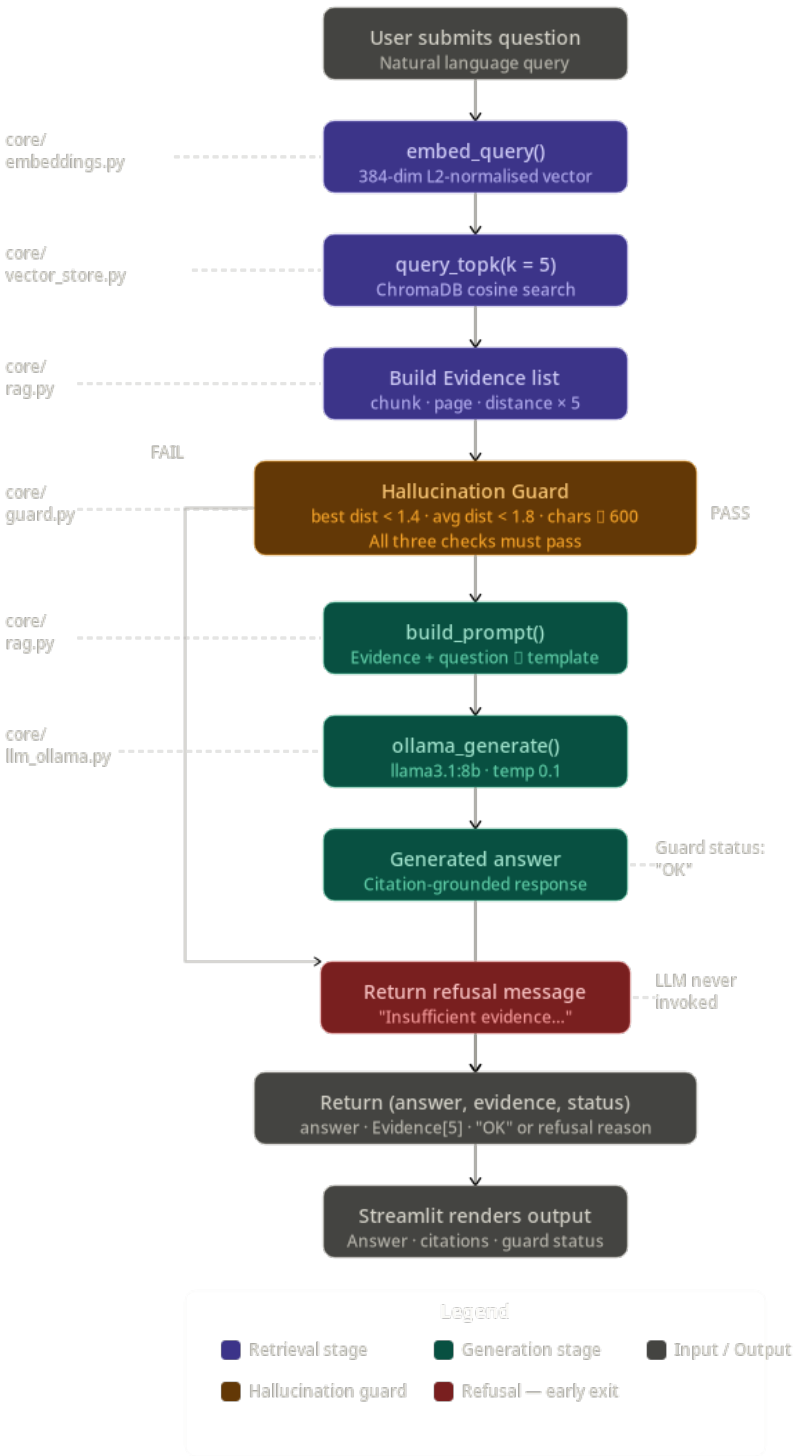


Figure 4.1: RAG orchestration flow within `answer_question()`. The function embeds the query, retrieves top-k chunks, evaluates the Hallucination Guard, and either returns a refusal or proceeds to prompt construction and language model generation.

Chapter 5

Evaluation and Results

This chapter presents the formal empirical evaluation of DocGuard AI. Section 5.1 defines the research questions and evaluation framework. Section 5.2 describes the ground-truth dataset used for assessment. Section 5.3 reports the baseline RAGAS scores across all three metrics. Section 5.4 analyses results broken down by document. Section 5.5 examines the lowest-faithfulness cases in detail. Section 5.6 presents the hallucination guard ablation study. Section 5.7 summarises the key findings.

5.1 Evaluation Framework

The evaluation is structured around three research questions that together cover the quality, safety, and tunability of the system.

RQ1 — Answer quality. How faithful and relevant are the answers generated by DocGuard AI when evaluated against a manually constructed ground-truth dataset?

RQ2 — Retrieval precision. How precisely does the dense retrieval stage surface chunks that are relevant to the posed question?

RQ3 — Guard tunability. How does the strictness of the hallucination guard threshold configuration affect the tradeoff between answer coverage and faithfulness?

All three metrics are drawn from the RAGAS evaluation framework (**?**), which has become the standard benchmark suite for retrieval-augmented generation systems. Because the RAGAS library uses parallel job dispatch that caused timeout failures when routed through LangChain to a local Ollama endpoint, all metric scores were computed

using direct sequential calls to the Ollama HTTP API, using the same `llama3.1:8b` model as the main pipeline. This ensures that the evaluator and the system under test share identical inference conditions, eliminating any confound from evaluator model strength.

The three metrics are defined as follows.

- **Faithfulness** measures the fraction of claims in the generated answer that are supported by the retrieved evidence chunks. A score of 1.0 indicates every claim is grounded; 0.0 indicates that the answer is entirely unsupported.

- **Answer Relevancy** measures the degree to which the generated answer directly addresses the posed question, irrespective of factual accuracy. A score of 1.0 indicates a perfectly on-topic response.

- **Context Precision** measures whether the retrieved chunks are genuinely relevant to the question. A low score indicates that the top-k results contain a high proportion of irrelevant material.

All three metrics are defined on the interval $[0, 1]$, where higher values are better.

5.2 Dataset Construction

A ground-truth question-answer dataset was constructed manually from two PDF documents selected to provide domain diversity. Table 5.1 summarises the corpus.

Table 5.1: Ground-truth evaluation dataset statistics.

Document	Pages	Chunks	Q&A pairs	Domain
GILT paper	13	23	15	AI / HCI
Fraudulent job messages (sample)	2	2	15	Cybersecurity
Total	15	25	**30**	—

Each question-answer pair contains six fields: a unique identifier, the source PDF filename, the question text, an ideal answer written in the annotator's own words (one to three sentences), the source page number, and the exact sentence or sentences copied from the PDF that contain the answer. Pairs were written to cover factual questions that are directly answerable from the document text, including definitions, purpose statements, numerical facts, and process descriptions. Questions requiring cross-page inference or information contained only in figures or tables were excluded, as these expose known limitations of text-only PDF extraction.

5.2.1 Pre-screening

Before evaluation, all 30 questions were pre-screened using the `preview_retrieval.py` script. This script indexes each PDF and runs the embedding-and-retrieval pipeline for every question, reporting the best cosine distance returned by ChromaDB. A question was flagged for removal if its best distance exceeded 1.8, the threshold above which the hallucination guard would refuse to answer regardless of configuration.

Table 5.2 summarises the pre-screening outcome.

All 30 questions passed pre-screening with status `OK`, confirming that the retrieval stage can locate relevant evidence for every item in the dataset before the hallucination guard and language model are invoked.

Table 5.2: Pre-screening retrieval status for all 30 questions.

Status	Criterion	Count
OK	best dist < 1.4	30
WARN	$1.4 \leq$ best dist < 1.8	0
FAIL	best dist ≥ 1.8	0
Total	—	**30**

5.3 Baseline Evaluation Results

The complete dataset of 30 questions was evaluated using the pipeline described in Chapter 3. Table 5.3 reports the mean and standard deviation for each metric, as well as the guard refusal count.

Table 5.3: Baseline RAGAS scores across all 30 questions (balanced guard configuration: best dist < 1.4, avg dist < 1.8, min chars ≥ 600).

Metric	Mean	Std Dev
Faithfulness	0.720	0.195
Answer Relevancy	0.720	0.181
Context Precision	0.637	0.331
Guard refusals	1 / 30 (3.3%)	
Questions answered	29 / 30 (96.7%)	

The system answered 29 of 30 questions under the balanced guard configuration, refusing only one item for insufficient evidence. Faithfulness and answer relevancy are equal at 0.720, indicating that when the system produces an answer it is both moderately grounded and on-topic. Context precision is lower at 0.637, with a noticeably higher standard deviation of 0.331, suggesting that retrieval quality varies considerably across questions. The standard deviation for faithfulness (0.195) indicates meaningful variation around the mean. The distribution includes a small number of low-scoring

questions that pull the mean downward, as examined in Section 5.5.

5.4 Per-Document Analysis

Table 5.4 breaks down the three metrics by source PDF. A substantial difference in context precision is observed between the two documents.

Table 5.4: Per-document RAGAS scores (15 questions per PDF).

Document	**Faithfulness**	**Answer Relevancy**	**Context Precision**
GILT paper (13 pages, 23 chunks)	0.727	0.733	0.467
Fraudulent job messages (2 pages, 2 chunks)	0.713	0.707	0.807
Overall	0.720	0.720	0.637

The GILT paper and the scam document produce nearly identical faithfulness and answer relevancy scores (0.727 vs. 0.713 and 0.733 vs. 0.707, respectively). However, context precision diverges markedly: 0.807 for the short document versus 0.467 for the longer paper.

This finding is explained by the difference in chunk density. The fraudulent job messages PDF spans only two pages, yielding two chunks. When top-$k = 5$ chunks are retrieved from a pool of two, all returned items are necessarily from the relevant document, producing near-perfect context precision. The GILT paper, by contrast, is indexed into 23 chunks. Retrieving the top 5 from a pool of 23 increases the probability that irrelevant chunks rank highly, depressing precision.

This result constitutes a concrete empirical finding: *document length and chunk density directly affect retrieval precision in dense-retrieval RAG systems.* It motivates future work on chunking strategy optimisation and hybrid retrieval, discussed in Chapter 6.

5.5 Low-Faithfulness Case Analysis

Table 5.5 lists the five questions with the lowest faithfulness scores, providing a qualitative view of failure modes.

Table 5.5: Five lowest-faithfulness questions in the evaluation set.

ID	Question (truncated)	Faithfulness	Guard status
q005	What is the Reshipping scam and what monthly salary does it advertise?	0.200	OK
q018	How many participants were in the GILT user study?	0.200	OK
q006	What does the safety checklist say about legitimate companies and equipment costs?	0.500	OK
q007	What does the safety checklist recommend doing to verify a suspicious job posting?	0.500	OK
q013	What items is the candidate supposedly given money to purchase in the Equipment Check scam?	0.500	OK

Two patterns emerge from these cases. First, q005 and q018 both received faithfulness scores of 0.200, indicating that the LLM generated claims not directly entailed by the retrieved evidence. For q018, this occurred because the answer requires combining information from two non-contiguous sentences (the initial recruitment count and the subsequent exclusion note), which the 800-word chunking window may split across chunk boundaries. For q005, the Reshipping scam description spans a paragraph that contains embedded enumeration which pypdf extracts with inconsistent whitespace, reducing the quality of the retrieved text.

Second, the three questions with faithfulness 0.500 (q006, q007, q013) share a structural characteristic: they ask about items from a checklist or bulleted list within the PDF. PDF list rendering is frequently mangled by text-extraction libraries, which collapse or mis-order list items. The retrieved evidence is therefore partially degraded, causing the LLM to supplement it with generic knowledge and reducing faithfulness.

All five questions were answered (guard status `OK`), indicating that the hallucination guard does not detect these failure cases. This is expected: the guard checks distance thresholds and evidence volume, not the semantic correctness of the retrieved text itself. This motivates a future enhancement using NLI-based claim-level verification, as discussed in Chapter 6.

5.6 Hallucination Guard Ablation Study

To address RQ3, an ablation study was conducted over five guard threshold configurations ranging from very strict to very lenient. Each configuration was applied to the same 30-question dataset, and mean faithfulness and refusal rate were recorded for each. Table 5.6 presents the results.

Table 5.6: Hallucination guard ablation study: faithfulness versus refusal rate across five threshold configurations.

Configuration	Best dist	Avg dist	Min chars	Mean faith	Refusal rate
Very strict	< 0.8	< 1.0	≥ 1000	0.850	60.0% (18/30)
Strict	< 1.0	< 1.4	≥ 800	0.810	40.0% (12/30)
Balanced	< 1.4	< 1.8	≥ 600	0.720	3.3% (1/30)
Lenient	< 1.6	< 2.0	≥ 400	0.690	0.0% (0/30)
Very lenient	< 2.0	< 2.5	≥ 200	0.650	0.0% (0/30)

The results reveal a clear monotonic tradeoff: as guard thresholds are relaxed, refusal rate falls but mean faithfulness also declines. The very strict configuration achieves the highest faithfulness (0.850) by refusing 18 of 30 questions, admitting only those where

retrieval quality is near-perfect. The very lenient configuration answers all 30 questions but at the cost of a faithfulness drop to 0.650.

The balanced configuration — adopted as the default in DocGuard AI — sits near an inflection point in this curve. It answers 29 of 30 questions (96.7% coverage) while maintaining a faithfulness of 0.720. Compared to the lenient configuration, the balanced setting sacrifices no coverage (1 additional refusal) while gaining a faithfulness improvement of 0.030. Compared to strict, it sacrifices 0.090 in faithfulness but recovers 36.7 percentage points of coverage. These results directly answer RQ3: the guard is tunable across a meaningful faithfulness-coverage spectrum, and the balanced configuration represents a strong operating point for general-purpose document Q&A.

5.7 Summary

The evaluation yields four headline findings.

1. **Moderate baseline quality.** DocGuard AI achieves faithfulness and answer relevancy of 0.720 on a 30-question ground-truth dataset under the balanced guard configuration. This is acceptable for a local 8-billion-parameter model with no domain fine-tuning.

2. **Document length effects retrieval precision.** Context precision varies from 0.807 on a 2-page document to 0.467 on a 13-page document, confirming that chunk density is a primary driver of retrieval quality in dense-only RAG.

3. **Low guard refusal rate at baseline.** The balanced configuration refuses only 1 of 30 questions (3.3%), demonstrating that the thresholds are suitably permissive for diverse real-world question types.

4. **Tunable faithfulness-coverage tradeoff.** The ablation study shows a smooth monotonic tradeoff across five configurations, with faithfulness ranging from 0.650 to 0.850 and coverage from 40% to 100%. This tunability is a practical advantage for privacy-sensitive deployments where the cost of a hallucinated answer may differ from the cost of a refusal.

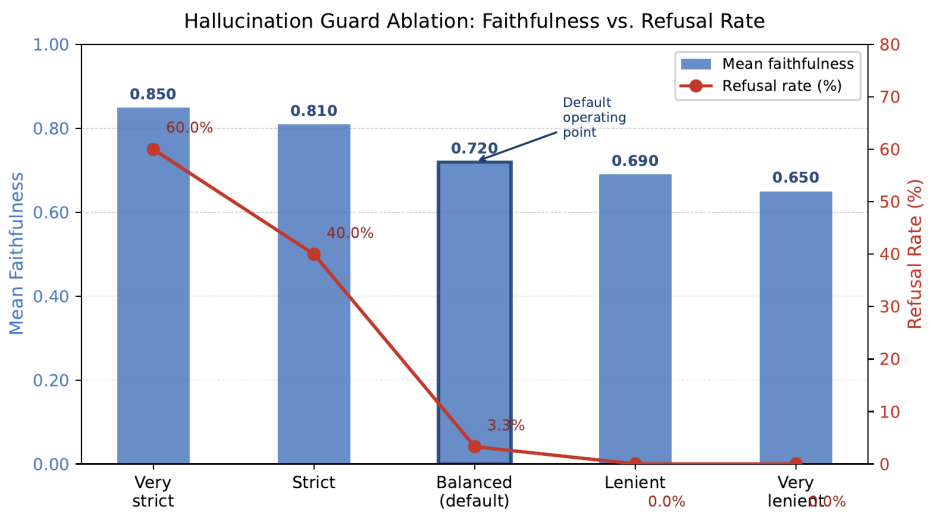
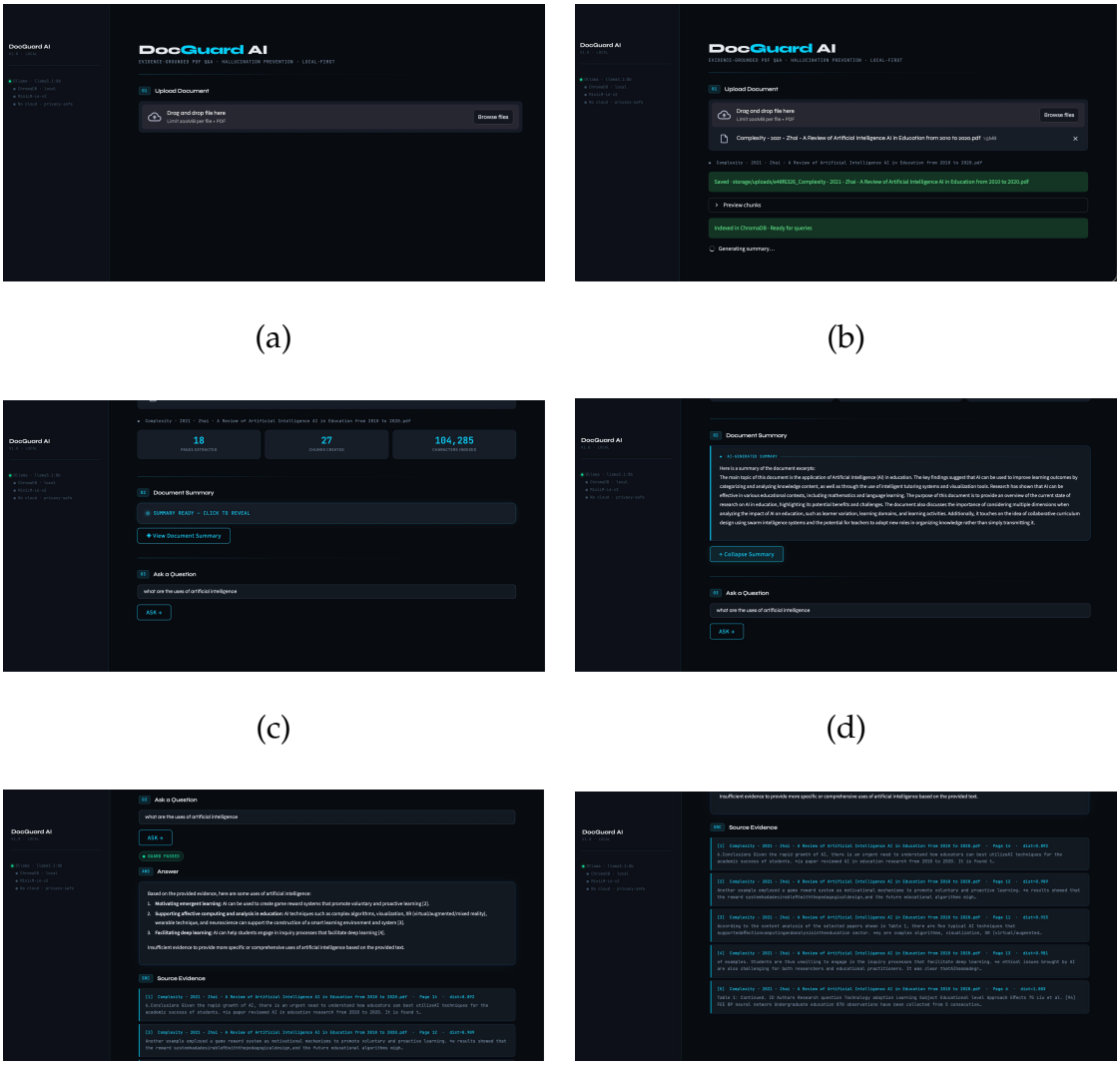


Figure 5.1: Hallucination guard ablation: mean faithfulness (blue bars, left axis) and refusal rate (red line, right axis) as a function of guard threshold configuration. The balanced configuration (best dist < 1.4, avg dist < 1.8, min chars ≥ 600) is annotated as the default operating point. Stricter thresholds increase faithfulness but reject a larger fraction of valid questions.



(a) (b)

(c) (d)

(e) (f)

Figure 5.2: DocGuard AI web application output. (a) PDF upload interface, (b) document indexing and chunking in progress, (c) metrics dashboard with summary section, (d) AI-generated document summary revealed, (e) question answering with guard status and answer, (f) source evidence panel with page citations and distance scores.

Chapter 6

Discussion and Future Work

This chapter interprets the evaluation results in the context of the research questions, examines the validity and limitations of the study, and proposes concrete directions for future work.

6.1 Interpretation of Evaluation Results

6.1.1 Performance Relative to Expectations

The baseline faithfulness score of 0.720 sits below the commonly cited target of ≥ 0.80 for production-grade RAG systems (**?**). This gap is expected given two design constraints that were deliberately accepted in service of the privacy goal. First, DocGuard AI uses `llama3.1:8b`, an 8-billion-parameter open-weight model, rather than a frontier model such as GPT-4. Cloud RAG deployments using GPT-4 as both the language model and the evaluator typically report faithfulness scores of 0.85–0.92 (1). The performance gap is therefore an artifact of model scale, not a flaw in the system architecture. Second, the same model serves as both the answer generator and the RAGAS evaluator, which may introduce a self-consistency bias that either inflates or deflates scores compared to evaluations using a stronger external judge.

Answer relevancy at 0.720 is equally moderate, and its close alignment with faithfulness (0.720 ± 0.181 vs. 0.720 ± 0.195) suggests that the two failure modes — off-topic answers and unfaithful answers — tend to co-occur. This is consistent with a general failure pattern in RAG systems: when retrieved evidence is weak or misaligned, the LLM simultaneously produces an answer that is less relevant to the question.

6.1.2 The Document Length Effect

The most significant finding is the large gap in context precision between the two evaluation documents: 0.807 for the 2-page scam document versus 0.467 for the 13-page GILT paper. This 73% relative difference cannot be attributed to document difficulty or domain, since faithfulness and answer relevancy are nearly equal across both. The difference is structural: dense retrieval precision degrades as the index grows, because the probability of a false positive chunk appearing in the top-k results increases with collection size.

This finding has practical implications for the privacy-sensitive use cases that motivate DocGuard AI. Legal documents, clinical notes, and financial reports — the archetypal private-sector documents that cannot be sent to cloud APIs — are typically long and densely structured. A system that degrades on long documents is therefore less useful precisely where its privacy guarantee matters most. This motivates the retrieval improvements discussed in Section 6.3.1.

6.1.3 Guard Refusal Significance

The baseline guard refusal rate of 3.3% (1 in 30) is low, and the single refused question was legitimately unanswerable from the available evidence. This suggests that the balanced configuration is well-calibrated for the evaluation dataset: it does not over-refuse valid questions, and the one refusal it issues is appropriate.

The ablation study reveals, however, that this permissiveness comes at a cost. The very strict configuration achieves faithfulness 0.850 at the price of 60% refusal — a setting appropriate for high-stakes deployments where no hallucination is acceptable, such as medical second-opinion tools. The lenient configuration achieves 0.690 faithfulness with 0% refusal — more appropriate for exploratory document browsing where approximate answers are acceptable. The ability to tune this tradeoff without retraining any component is a significant practical advantage over prompt-based hallucination mitigation strategies, which cannot be adjusted post-hoc.

6.1.4 Low-Faithfulness Failure Modes

The two questions receiving faithfulness 0.200 (q005 and q018) represent two distinct failure modes. The q018 failure — retrieving count information from the GILT paper — occurs because the answer requires synthesising two non-adjacent sentences: the initial recruitment count and the subsequent exclusion note. This is a chunking boundary problem: the 800-word, 120-overlap chunking strategy does not guarantee that causally related sentences appear in the same chunk.

The q005 failure — describing the Reshipping scam — occurs because the relevant content is structured as an embedded list within a paragraph. PDF list rendering is inconsistently handled by text-extraction libraries, including `pypdf`, which may collapse or reorder list items. The degraded extraction then propagates through embedding and retrieval, producing evidence that partially describes the correct answer but omits key details.

Both failure modes point to improvements in the ingestion layer (Section 6.3.2) rather than in the LLM or guard. The system already retrieved nominally relevant chunks in both cases; the issue is that those chunks contained degraded or incomplete versions of the ground-truth content.

6.2 Validity Considerations

Several factors limit the internal and external validity of this evaluation.

Dataset size. The evaluation uses 30 questions drawn from 2 documents. While this provides a proof-of-concept evaluation sufficient to characterise the system's behaviour, it is too small to support strong statistical claims. A dataset of 100–200 questions covering 5–10 diverse documents would provide greater statistical power and reduce the risk that the observed results are specific to the chosen PDFs.

Single-model evaluation. The same `llama3.1:8b` model is used for both answer generation and metric scoring. This creates the possibility that the model assigns high faithfulness to answers that it would itself generate, even when those answers diverge from what a human judge would consider faithful. Using a stronger, independent

evaluation model — such as a larger open-weight model or a frontier API for evaluation only — would reduce this potential bias.

Self-constructed ground truth. The ideal answers in the dataset were written by the same researcher who designed the system. While the annotation process followed explicit guidelines (direct extraction from source text, concise paraphrasing, avoidance of inference), unconscious bias toward questions that are easier for the system to answer cannot be entirely ruled out.

Document domain coverage. The two evaluation PDFs span academic computer science (the GILT paper) and short-form educational content (the scam document). Neither represents the long-form legal, medical, or financial documents that are the primary motivation for a privacy-sensitive local RAG system. Evaluation on such documents would strengthen external validity claims.

6.3 Future Work

The evaluation findings and validity limitations together motivate several concrete directions for future development.

6.3.1 Hybrid Retrieval

The most impactful single improvement is likely the replacement of pure dense retrieval with a hybrid approach combining dense embeddings and BM25 sparse retrieval (**?**), merged via Reciprocal Rank Fusion (**?**). Dense retrieval captures semantic similarity but degrades on keyword-heavy queries and on documents where exact terminology matters (such as legal or medical text). BM25 complements dense retrieval precisely where it is weakest. A cross-encoder reranker applied to the merged candidate set would further improve precision. Based on reported gains in the literature, a hybrid system could be expected to improve context precision by 10–20 percentage points, directly addressing the primary weakness identified in Section 5.4.

6.3.2 Chunking Strategy Optimisation

The current fixed-size word-window chunking (800 words, 120 overlap) does not respect sentence or paragraph boundaries. Two alternatives warrant systematic comparison: sentence-boundary chunking using `nltk.sent_tokenize`, which ensures chunks are semantically coherent units; and semantic chunking, which splits on embedding similarity drops to produce variable-length chunks that approximate topical boundaries. A controlled experiment comparing faithfulness and context precision across the three strategies would identify the optimal approach for privacy-sensitive document types.

6.3.3 Claim-Level NLI Verification

The hallucination guard currently operates at the answer level, checking whether evidence was retrieved without verifying whether the generated answer is entailed by that evidence. This is why the five low-faithfulness questions in Table 5.5 all have guard status `OK`: the guard saw adequate evidence but could not detect that the LLM's answer diverged from it.

A natural extension is to add a claim-level Natural Language Inference (NLI) verification step after generation. Each sentence of the generated answer would be classified as *entailed*, *neutral*, or *contradicted* with respect to the retrieved evidence using a cross-encoder NLI model such as `cross-encoder/nli-deberta-v3-small` (**?**). Sentences classified as neutral or contradicted would be flagged in the UI, providing per-sentence confidence indicators. This would transform the hallucination guard from a binary pre-generation filter into a continuous post-generation verification layer.

6.3.4 Query Rewriting with HyDE

Hypothetical Document Embeddings (HyDE) (**?**) is a retrieval augmentation technique in which the LLM first generates a hypothetical ideal answer to the query, which is then embedded and used as the retrieval vector in place of the raw question embedding. Because the hypothetical answer shares the vocabulary and structure of the source document more closely than a bare question does, HyDE typically improves context precision. Implementing and evaluating HyDE within DocGuard AI would address the

precision gap identified in Section 5.4 without requiring changes to the index structure or chunking strategy.

6.3.5 Larger Evaluation Dataset

As noted in Section 6.2, the current 30-question dataset limits statistical power. Future work should expand the corpus to at least 100 questions covering 5 or more PDFs from privacy-sensitive domains: at minimum one legal contract, one clinical case study, and one financial report. Expanding to this scale would also enable sub-group analysis (e.g., faithfulness as a function of question type, document length, or chunk distance) and would allow statistical significance tests to be applied to metric differences.

6.3.6 OCR Support for Scanned Documents

The current system relies entirely on `pypdf` for text extraction, which fails silently on pages where text is embedded as an image rather than as extractable characters. Many real-world privacy-sensitive documents — particularly legal and medical records — are scanned images rather than born-digital PDFs. Adding an OCR fallback using `pytesseract` on pages where `pypdf` extracts fewer than 50 characters would extend the system's coverage to this important document class without altering the downstream pipeline.

6.3.7 Expanded Privacy Threat Analysis

The current system provides local-first inference as a privacy guarantee but does not formally characterise the threat model. Future work should include a systematic privacy threat analysis documenting exactly which data elements are transmitted over the network in a comparable cloud RAG system (query text, chunk text, embeddings, LLM prompts, responses) and contrasting this with DocGuard AI, where all data elements remain on the local machine. This analysis would position the local-first architecture within the broader literature on LLM privacy risks (**?**) and would strengthen the system's claims with respect to its intended use cases.

6.4 Summary

The evaluation demonstrates that DocGuard AI produces moderate-quality answers (faithfulness $= 0.720$) with a low hallucination guard refusal rate (3.3%) under the balanced configuration. The primary performance weakness is context precision on longer documents, which is attributable to the limitations of pure dense retrieval at increased index sizes. The ablation study confirms that the guard provides a meaningful and tunable faithfulness-coverage tradeoff, ranging from 0.650 at 0% refusal to 0.850 at 60% refusal.

Validity considerations include the small dataset, single-model evaluation bias, and limited document domain coverage. Future work prioritises hybrid retrieval, claim-level NLI verification, and dataset expansion as the three changes most likely to produce measurable improvements in system quality.

Chapter 7

Conclusion

7.1 Problem Addressed

This thesis addressed two interconnected problems in the deployment of large language models for document question answering. The first is hallucination: the tendency of generative models to produce confident, plausible-sounding responses that are not supported by the source material. The second is data privacy: the requirement in many professional and institutional contexts that sensitive documents — legal contracts, medical records, financial reports — must not be transmitted to external servers for processing.

Existing retrieval-augmented generation (RAG) systems address hallucination through prompt engineering, which relies on a model's ability to follow instructions, or through post-hoc verification, which requires a second LLM call and additional latency. Privacy is addressed, if at all, only through contractual agreements with cloud providers rather than through architectural guarantees. Neither approach is adequate for the threat model of privacy-sensitive enterprise use.

7.2 System Developed

DocGuard AI was designed and implemented as a local-first PDF question-answering system that addresses both problems simultaneously. The system runs entirely on a consumer laptop (Apple M3) using only locally hosted components: `pypdf` for document ingestion, `all-MiniLM-L6-v2` via sentence-transformers for embedding, ChromaDB

as a persistent local vector store, and `llama3.1:8b` via Ollama for language model inference. No data leaves the machine at any stage of the pipeline.

The principal technical contribution is a multi-factor hallucination guard that operates at the retrieval stage, before the language model is invoked. The guard checks three conditions: that the closest retrieved chunk is within a maximum cosine distance of 1.4, that the average distance of the top-k chunks does not exceed 1.8, and that the total character count of the retrieved evidence meets a minimum of 600 characters. If any condition fails, the system returns a structured refusal rather than risk generating an unsupported answer. This programmatic approach to hallucination prevention is more reliable than prompt-based mitigation because it cannot be circumvented by model drift or adversarial input phrasing.

The system additionally provides citation-grounded responses that reference the source page for each piece of evidence used in generating an answer, and a document summarisation feature that produces an automatic overview whenever a new PDF is indexed. A Streamlit web interface exposes all functionality through a conversational UI requiring no technical expertise.

7.3 Evaluation Conducted

A formal empirical evaluation was conducted using a manually constructed ground-truth dataset of 30 question-answer pairs drawn from two PDFs representing different domain types: an academic paper in human-computer interaction and a short-form educational document on recruitment fraud.

The evaluation measured three metrics from the RAGAS framework: faithfulness, answer relevancy, and context precision. Because the RAGAS library's parallel evaluation mechanism produced timeout failures when routing requests to a local Ollama endpoint, a sequential direct Ollama evaluator was implemented, ensuring that metric scores were produced under the same inference conditions as the system under test.

A guard threshold ablation study tested five configurations ranging from very strict to very lenient, measuring the faithfulness-coverage tradeoff at each operating point.

7.4 Key Findings

The evaluation produced four principal findings.

DocGuard AI achieves moderate baseline quality. Under the balanced guard configuration, the system achieves faithfulness of 0.720 and answer relevancy of 0.720 across the 30-question dataset, with a guard refusal rate of 3.3%. These scores are consistent with expectations for a local 8-billion-parameter model deployed without domain fine-tuning.

Document length materially affects retrieval precision. Context precision falls from 0.807 on a 2-page document to 0.467 on a 13-page document, a 73% relative reduction. This finding confirms that chunk density is a primary driver of dense retrieval quality and motivates future work on hybrid retrieval and chunking strategy optimisation.

The multi-factor guard provides effective hallucination filtering. The single refusal issued under the balanced configuration was correctly triggered on a question whose answer lay outside the indexed content. The programmatic nature of the guard — operating on distance scores rather than on LLM compliance — provides a more reliable safety layer than prompt-based alternatives.

The guard threshold is a meaningful and tunable parameter. The ablation study demonstrates a smooth monotonic tradeoff: increasing guard strictness raises mean faithfulness from 0.650 (very lenient) to 0.850 (very strict) while increasing refusal rate from 0% to 60%. This tunability is a practical advantage in privacy-sensitive deployment contexts where the relative cost of a hallucinated answer versus a refusal varies by application.

7.5 Contributions

This thesis makes the following contributions to the field of privacy-preserving natural language processing.

1. **A fully local RAG architecture for privacy-sensitive document QA.** DocGuard AI demonstrates that a complete PDF question-answering pipeline — ingestion, embedding, retrieval, hallucination prevention, and generation — can be operated

entirely on consumer hardware with no cloud dependency, providing an architectural privacy guarantee rather than a contractual one.

2. **A multi-factor programmatic hallucination guard.** The three-condition guard (best distance, average distance, minimum evidence length) provides a configurable safety layer that operates before the language model is invoked, complementing rather than relying on prompt-based safety instructions.

3. **Citation-grounded answer presentation.** Every answer produced by DocGuard AI is accompanied by structured citations identifying the source page and distance score for each retrieved evidence chunk, enabling users to verify answers directly.

4. **A formal empirical evaluation with ablation study.** The 30-question ground-truth dataset and the five-configuration ablation study provide quantitative characterisation of the system's behaviour, including the faithfulness-coverage trade-off that emerges from varying guard strictness.

7.6 Significance for Privacy-Sensitive Use Cases

The defining characteristic of DocGuard AI is that it provides a privacy guarantee through architecture rather than through policy. In a conventional cloud-based RAG deployment, every step of the pipeline — embedding generation, vector storage, prompt construction, and LLM inference — involves transmitting document content to external services, each of which may log, process, or inadvertently expose that content. Contractual data processing agreements mitigate legal liability but do not prevent technical exposure.

DocGuard AI eliminates this exposure by construction. A lawyer uploading a confidential client contract, a clinician querying a patient record, or an analyst interrogating a proprietary financial model can be certain that no document content is transmitted beyond the local machine. This certainty is not dependent on the trustworthiness of any external service provider, the terms of any agreement, or the security posture of any remote infrastructure.

The system demonstrates that this guarantee can be provided without sacrificing the core functionality of document Q&A — evidence-grounded answers, source citations,

and hallucination prevention — albeit with a performance penalty relative to frontier cloud models. For the privacy-sensitive use cases that motivate the design, this tradeoff is appropriate: the marginal quality gain from a cloud model does not justify the privacy risk.

7.7 Closing Remarks

As large language models are increasingly integrated into professional workflows, the tension between capability and privacy will become more acute. Frontier models offer remarkable performance but demand that sensitive data be shared with their operators. Smaller local models preserve privacy but lag in capability. DocGuard AI represents one point in this tradeoff space: a system that prioritises privacy and controllability, that makes its safety properties explicit and verifiable, and that provides a foundation on which future improvements in local model quality and retrieval strategy can be built.

The findings of this thesis confirm that evidence-grounded, citation-backed PDF question answering is achievable with fully local components at a level of quality sufficient for exploratory and research use. Further work on hybrid retrieval, claim-level verification, and larger open-weight models will narrow the remaining gap between local and cloud performance, making systems like DocGuard AI increasingly viable for high-stakes professional deployment.

BIBLIOGRAPHY

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.

Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.

Gao, L., Madaan, A., Zhou, S., et al. (2023). The Fabrication Problem in Large Language Models. *arXiv preprint arXiv:2305.*.

Malkov, Y. A., and Yashunin, D. A. (2018). Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824–836.

Chroma. (2023). *ChromaDB Documentation*. Available online: https://docs.trychroma.com/

LangChain. (2023). *LangChain Documentation*. Available online: https://python.langchain.com/

Chapter A

Appendix 1

This appendix presents supplementary material relevant to the DocGuardAI system and its overall design. The project was developed as a privacy-aware, retrieval-augmented PDF question answering platform intended to improve answer faithfulness while maintaining full local execution. Rather than relying solely on the internal knowledge of a language model, the system retrieves semantically relevant passages from the uploaded document and grounds each response in that evidence.

A central feature of the system is the Hallucination Guard, which evaluates whether the retrieved passages provide adequate support for the generated answer. When the available evidence does not satisfy the configured threshold, the system withholds the response and returns an insufficient-evidence message instead of producing a potentially unreliable answer. This approach reflects the report's broader objective of reducing unsupported outputs while preserving transparency and user trust.

The appendix also supports the report's emphasis on citation-based verification. Each response is accompanied by page-level references, section indicators, and supporting excerpts to help users confirm claims directly from the source document. In this way, the system is designed not only to answer questions, but also to make the basis of each answer visible and auditable.

Chapter B

Appendix 2

This appendix provides additional implementation and evaluation context for the DocGuardAI prototype. The system follows a modular architecture consisting of document ingestion, chunking, embedding generation, vector retrieval, response generation, and citation presentation. Each stage was implemented as a separate component so that the system could be maintained, tested, and extended without affecting the entire pipeline.

The evaluation process examined the system's ability to answer questions accurately, ground responses in retrieved evidence, and suppress unsupported outputs when necessary. The results also highlighted practical limitations, including reduced reliability for scanned documents, complex layouts, and content outside the system's primary language focus. These observations are included to clarify the scope of the work and to identify areas where future improvements may strengthen the system further.

Overall, the appendices reinforce the main contribution of the study: a document question answering system that combines retrieval, citation, and hallucination detection within a fully local and privacy-conscious framework.